In [ ]:

Porter Delivery time estimation

Porter is India's Largest Marketplace for Intra-City Logistics. Leader in the country's $40 billion intra-city logistics market, Porter strives to improve the lives of 1,50,000+ driver-partners by providing them with consistent earning & independence. Currently, the company has serviced 5+ million customers

Porter works with a wide range of restaurants for delivering their items directly to the people.

Porter is India's Largest Marketplace for Intra-City Logistics. Leader in the country's $40 billion intra-city logistics market, Porter strives to improve the lives of 1,50,000+ driver-partners by providing them with consistent earning & independence. Currently, the company has serviced 5+ million customers

Porter works with a wide range of restaurants for delivering their items directly to the people.

In [1]:
```
import pandas as pd
import numpy as np
```

Each row in this file corresponds to one unique delivery. Each column corresponds to a feature as explained below.

- **market_id**: integer id for the market where the restaurant lies
- **created_at**: the timestamp at which the order was placed
- **actual_delivery_time**: the timestamp when the order was delivered
- **store_primary_category**: category for the restaurant
- **order_protocol**: integer code value for order protocol (how the order was placed, i.e., through porter, call to restaurant, pre-booked, third party, etc.)
- **total_items** : total items
- **subtotal**: final price of the order
- **num_distinct_items**: the number of distinct items in the order
- **min_item_price**: price of the cheapest item in the order
- **max_item_price**: price of the costliest item in order
- **total_onshift_partners**: number of delivery partners on duty at the time the order was placed
- **total_busy_partners**: number of delivery partners attending to other tasks

- **total_outstanding_orders**: total number of orders to be fulfilled at the moment
- **estimated_store_to_consumer_driving_duration**: approximate travel time from restaurant to customer

In [2]:
```
df = pd.read_csv('/content/drive/Othercomputers/My Laptop/Desktop/SCALER-20240
df
```

Out[2]:

	market_id	created_at	actual_delivery_time	store_primary_category	or
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	
...	...	...	...	...	
175772	1.0	2015-02-17 00:19:41	2015-02-17 01:02:41	28	
175773	1.0	2015-02-13 00:01:59	2015-02-13 01:03:59	28	
175774	1.0	2015-01-24 04:46:08	2015-01-24 05:32:08	28	
175775	1.0	2015-02-01 18:18:15	2015-02-01 19:03:15	58	
175776	1.0	2015-02-08 19:24:33	2015-02-08 20:01:33	58	

175777 rows × 14 columns

Examine dataset structure, characteristics, and statistical summary

In [3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
 #   Column                                        Non-Null Count   Dtype  
---  ------                                        --------------   -----  
 0   market_id                                     175777 non-null  float64
 1   created_at                                    175777 non-null  object 
 2   actual_delivery_time                          175777 non-null  object 
 3   store_primary_category                        175777 non-null  int64  
 4   order_protocol                                175777 non-null  float64
 5   total_items                                   175777 non-null  int64  
 6   subtotal                                      175777 non-null  int64  
 7   num_distinct_items                            175777 non-null  int64  
 8   min_item_price                                175777 non-null  int64  
 9   max_item_price                                175777 non-null  int64  
 10  total_onshift_dashers                         175777 non-null  float64
 11  total_busy_dashers                            175777 non-null  float64
 12  total_outstanding_orders                      175777 non-null  float64
 13  estimated_store_to_consumer_driving_duration  175777 non-null  float64
dtypes: float64(6), int64(6), object(2)
memory usage: 18.8+ MB
```

In [4]:
```
df.describe()
```

Out[4]:

	market_id	store_primary_category	order_protocol	total_items	
count	175777.000000	175777.000000	175777.000000	175777.000000	17
mean	2.743726	35.887949	2.911752	3.204976	
std	1.330963	20.728254	1.513128	2.674055	
min	1.000000	0.000000	1.000000	1.000000	
25%	2.000000	18.000000	1.000000	2.000000	
50%	2.000000	38.000000	3.000000	3.000000	
75%	4.000000	55.000000	4.000000	4.000000	
max	6.000000	72.000000	7.000000	411.000000	2

Observations:

- By looking at the statistical description we can say columns like market_id,order_protocol,distinct_items are categorical, we will explore further in the case study will mark our conclusion and impute for machine learning model appropriately.
- min_item_price, total_onshift_dashers, total_busy_dashers, and total_outstanding_orders contain negative minimum values. These are impossible in a real-world context (e.g., negative dashers or item

prices) and indicate data errors that need to be cleaned or imputed.

In [5]:
```
df.columns
```

Out[5]:
```
Index(['market_id', 'created_at', 'actual_delivery_time',
       'store_primary_category', 'order_protocol', 'total_items', 'subtotal',
       'num_distinct_items', 'min_item_price', 'max_item_price',
       'total_onshift_dashers', 'total_busy_dashers',
       'total_outstanding_orders',
       'estimated_store_to_consumer_driving_duration'],
      dtype='object')
```

In [6]:
```
# unique categories
for i in df.columns:
  print(i , "-->", df[i].nunique())
```

```
market_id --> 6
created_at --> 162649
actual_delivery_time --> 160344
store_primary_category --> 73
order_protocol --> 7
total_items --> 54
subtotal --> 8182
num_distinct_items --> 20
min_item_price --> 2251
max_item_price --> 2585
total_onshift_dashers --> 172
total_busy_dashers --> 158
total_outstanding_orders --> 281
estimated_store_to_consumer_driving_duration --> 1318
```

In [7]:
```
for i in df.columns:
  print(
        df[i].value_counts())
```

```
market_id
2.0    53469
4.0    46222
1.0    37115
3.0    21075
5.0    17258
6.0      638
Name: count, dtype: int64
created_at
2015-02-11 19:50:43    6
2015-01-31 01:41:10    5
2015-02-08 02:20:03    5
2015-02-11 19:51:06    5
2015-01-24 01:56:33    5
                      ..
2015-01-23 01:15:37    1
2015-02-03 19:27:36    1
2015-02-05 01:05:42    1
2015-02-13 19:04:52    1
2015-02-02 20:07:44    1
Name: count, Length: 162649, dtype: int64
actual_delivery_time
2015-02-15 04:18:47    5
2015-02-12 03:28:44    5
2015-02-16 03:06:54    5
2015-02-14 03:01:28    5
2015-02-14 02:58:40    5
                      ..
2015-02-02 03:24:25    1
2015-02-06 04:27:34    1
2015-02-08 04:13:08    1
2015-02-07 03:37:36    1
2015-01-27 04:14:40    1
Name: count, Length: 160344, dtype: int64
store_primary_category
4     18183
55    15745
46    15586
13     9915
58     8995
      ...
1        10
43        9
8         2
21        1
3         1
Name: count, Length: 73, dtype: int64
order_protocol
1.0    48404
3.0    47125
5.0    41415
2.0    20890
4.0    17246
6.0      678
```

```
7.0       19
Name: count, dtype: int64
total_items
2      48979
1      35647
3      35144
4      22659
5      12586
6       7765
7       4511
8       2708
9       1668
10      1127
11       732
12       586
13       352
14       294
15       209
16       164
17       118
18        98
19        60
20        56
21        44
22        34
24        33
25        32
26        29
28        17
23        17
30        15
27        14
29        12
42         7
34         7
35         7
31         5
33         5
40         4
36         4
32         4
48         4
39         3
41         2
38         2
37         2
56         1
57         1
45         1
411        1
47         1
51         1
59         1
44         1
```

```
49         1
64         1
66         1
Name: count, dtype: int64
subtotal
1500     832
1800     770
1700     759
1095     758
2000     743
        ...
5709       1
6016       1
12430      1
5179       1
9956       1
Name: count, Length: 8182, dtype: int64
num_distinct_items
2     52718
1     43993
3     37435
4     20780
5     10537
6      5065
7      2611
8      1252
9       650
10      351
11      195
12       88
13       50
14       27
15       12
16        6
18        3
17        2
20        1
19        1
Name: count, dtype: int64
min_item_price
795     3899
250     3601
150     3362
200     3235
350     3001
        ...
2087       1
2131       1
2807       1
2585       1
1670       1
Name: count, Length: 2251, dtype: int64
max_item_price
1095    5089
```

```
995     4638
1195    3837
1295    3757
999     3402
        ...
3081       1
7210       1
3031       1
3043       1
4120       1
Name: count, Length: 2585, dtype: int64
total_onshift_dashers
 0.0      3541
 15.0     2831
 18.0     2830
 21.0     2758
 19.0     2744
          ...
 164.0       1
 159.0       1
 169.0       1
-4.0         1
 168.0       1
Name: count, Length: 172, dtype: int64
total_busy_dashers
 0.0      4066
 10.0     3017
 13.0     2938
 18.0     2926
 6.0      2914
          ...
-3.0         2
 152.0       2
 154.0       1
 149.0       1
-5.0         1
Name: count, Length: 158, dtype: int64
total_outstanding_orders
0.0      4003
9.0      2650
10.0     2598
8.0      2589
6.0      2556
         ...
264.0       1
283.0       1
245.0       1
273.0       1
260.0       1
Name: count, Length: 281, dtype: int64
estimated_store_to_consumer_driving_duration
512.0     353
549.0     347
603.0     343
```

```
563.0     333
590.0     333
          ...
1361.0      1
1246.0      1
1739.0      1
1298.0      1
1299.0      1
Name: count, Length: 1318, dtype: int64
```

In [7]:

In [8]:
```
# checking duplicates
df.duplicated().sum()
```

Out[8]:
```
np.int64(0)
```

There are no duplicates

In [8]:

In [9]:
```
df['created_at'] = pd.to_datetime(df['created_at'])
df['actual_delivery_time'] = pd.to_datetime(df['actual_delivery_time'])
```

In [10]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   created_at                                    175777 non-null  datetime6
4[ns]
 2   actual_delivery_time                          175777 non-null  datetime6
4[ns]
 3   store_primary_category                        175777 non-null  int64
 4   order_protocol                                175777 non-null  float64
 5   total_items                                   175777 non-null  int64
 6   subtotal                                      175777 non-null  int64
 7   num_distinct_items                            175777 non-null  int64
 8   min_item_price                                175777 non-null  int64
 9   max_item_price                                175777 non-null  int64
 10  total_onshift_dashers                         175777 non-null  float64
 11  total_busy_dashers                            175777 non-null  float64
 12  total_outstanding_orders                      175777 non-null  float64
 13  estimated_store_to_consumer_driving_duration  175777 non-null  float64
dtypes: datetime64[ns](2), float64(6), int64(6)
memory usage: 18.8 MB
```

Missing value

In [11]:
```
#checking missing values
df.isna().sum().sort_values(ascending = False)
```

Out[11]:

	0
market_id	0
created_at	0
actual_delivery_time	0
store_primary_category	0
order_protocol	0
total_items	0
subtotal	0
num_distinct_items	0
min_item_price	0
max_item_price	0
total_onshift_dashers	0
total_busy_dashers	0
total_outstanding_orders	0
estimated_store_to_consumer_driving_duration	0

dtype: int64

Observations:

- There are no null values

In [11]:

In [11]:

In [11]:

EDA

Univariate analysis

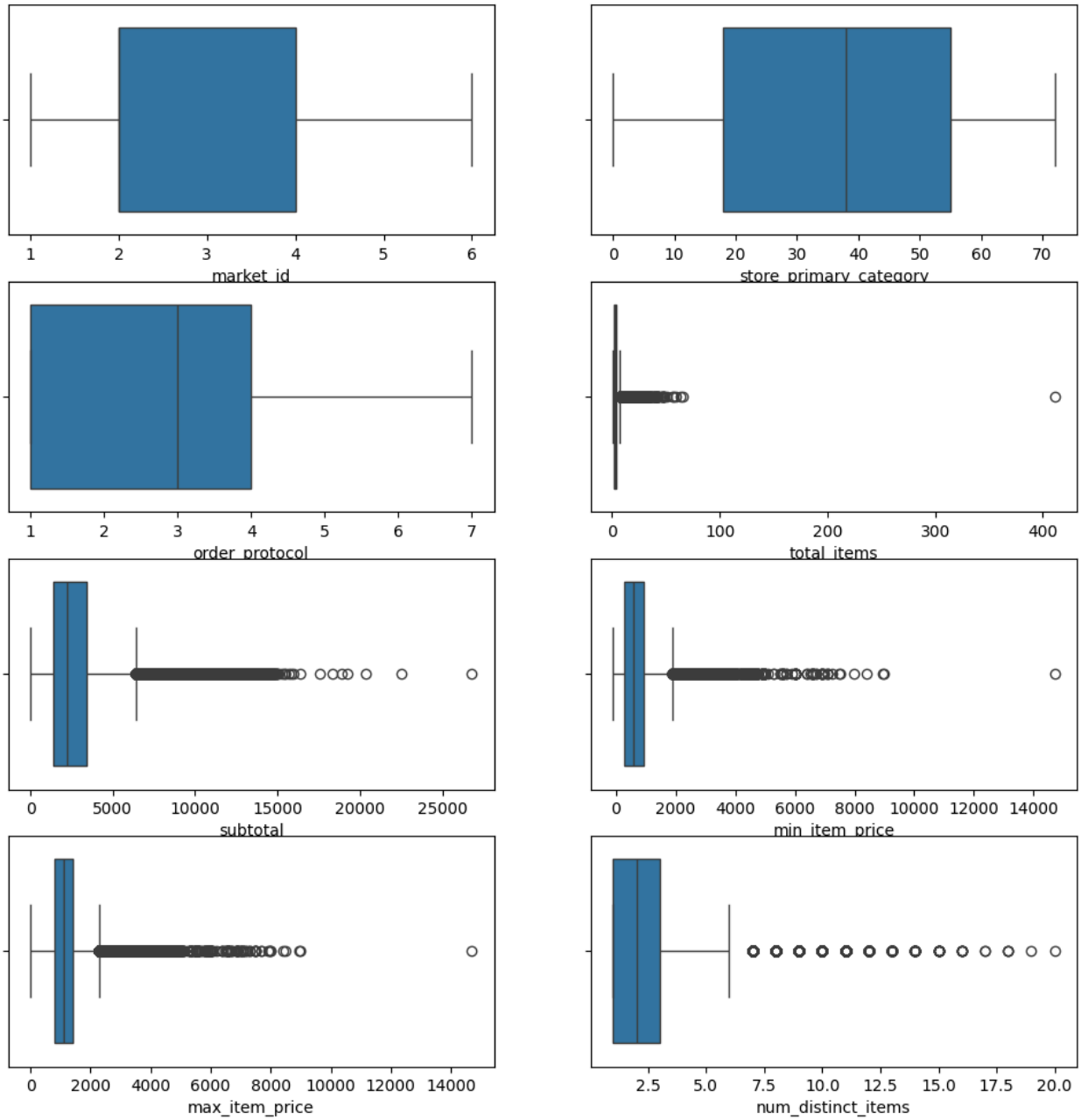
In [12]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [12]:
```

```

Boxplot

In [13]:
```
#market_id        store_primary_category        order_protocol        total_it
```

In [14]:
```
plt.figure(figsize=(12, 12))

plt.subplot(4,2,1)
sns.boxplot(x=df['market_id'])

plt.subplot(4,2,2)
sns.boxplot(x=df['store_primary_category'])

plt.subplot(4,2,3)
sns.boxplot(x=df['order_protocol'])

plt.subplot(4,2,4)
sns.boxplot(x=df['total_items'])

plt.subplot(4,2,5)
sns.boxplot(x=df['subtotal'])

plt.subplot(4,2,6)
sns.boxplot(x=df['min_item_price'])

plt.subplot(4,2,7)
sns.boxplot(x=df['max_item_price'])

plt.subplot(4,2,8)
sns.boxplot(x=df['num_distinct_items'])

plt.show()
```



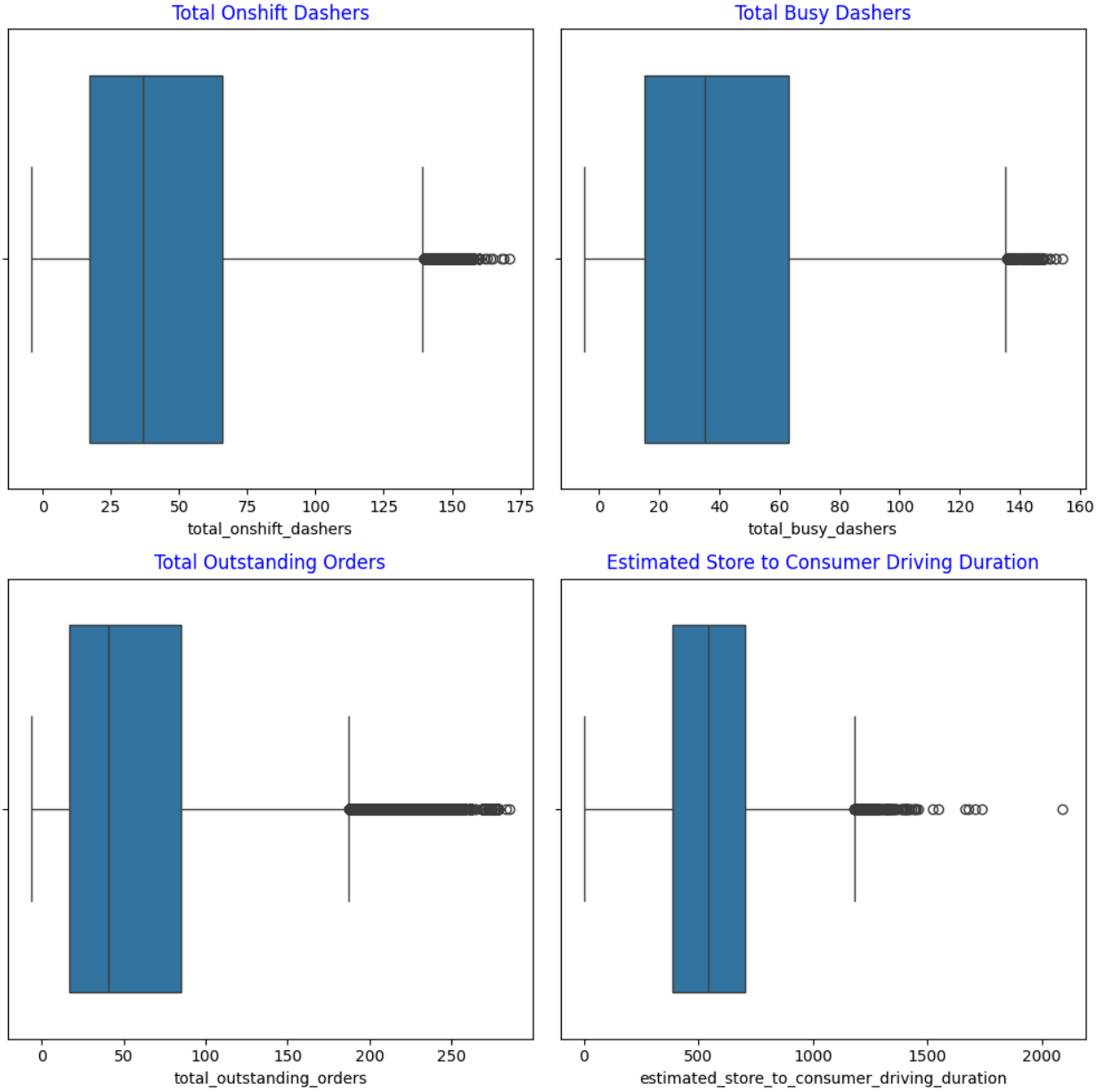
Observations:

- There seems to be many outlier and distribution seems to be right skewed and hence outlier treatment will be required.

In [14]:

In [15]:
```
plt.figure(figsize=(10, 10))

plt.subplot(2, 2, 1)
sns.boxplot(x=df['total_onshift_dashers'])
plt.title('Total Onshift Dashers', color='blue')
```

```
plt.subplot(2, 2, 2)
sns.boxplot(x=df['total_busy_dashers'])
plt.title('Total Busy Dashers', color='blue')

plt.subplot(2, 2, 3)
sns.boxplot(x=df['total_outstanding_orders'])
plt.title('Total Outstanding Orders', color='blue')

plt.subplot(2, 2, 4)
sns.boxplot(x=df['estimated_store_to_consumer_driving_duration'])
plt.title('Estimated Store to Consumer Driving Duration', color='blue')

plt.tight_layout()
plt.show()
```

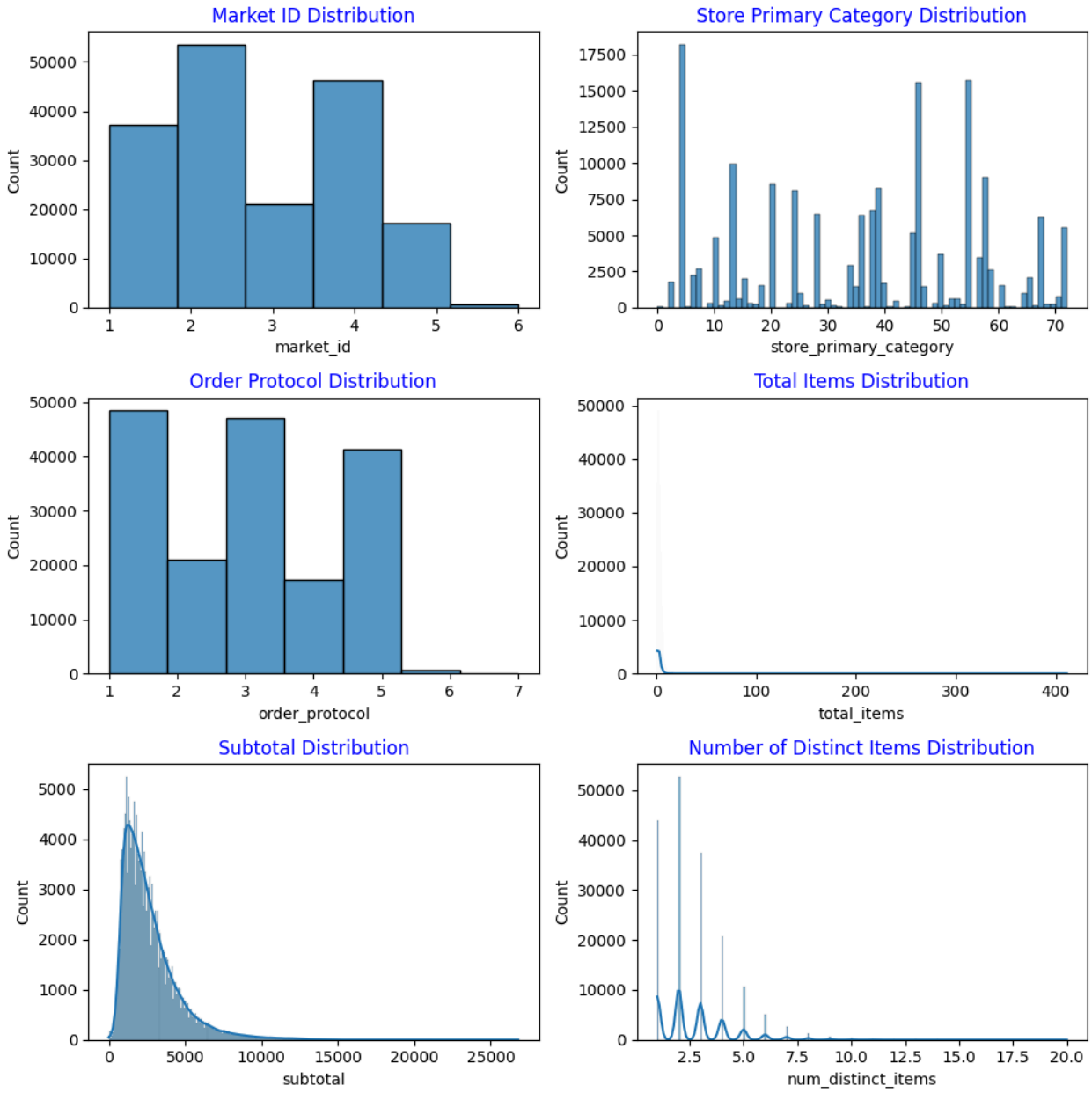


In [15]:

In [15]:

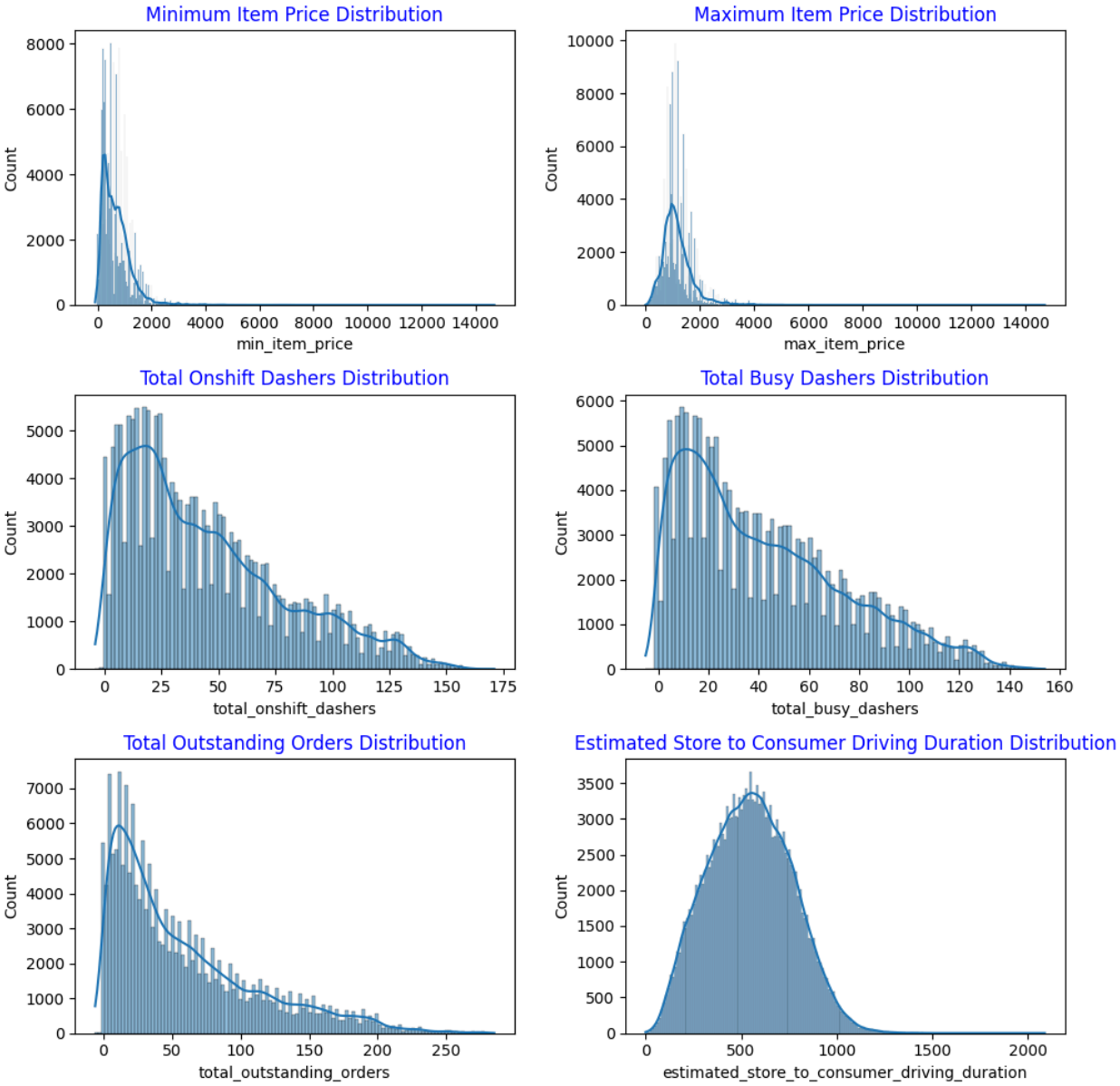
Histplot

In [16]:
```
plt.figure(figsize=(10, 10))

plt.subplot(3, 2, 1)
sns.histplot(df['market_id'], kde=False, bins=len(df['market_id'].unique()))
plt.title('Market ID Distribution', color='blue')

plt.subplot(3, 2, 2)
sns.histplot(df['store_primary_category'], kde=False, bins=len(df['store_prima
plt.title('Store Primary Category Distribution', color='blue')

plt.subplot(3, 2, 3)
sns.histplot(df['order_protocol'], kde=False, bins=len(df['order_protocol'].ur
plt.title('Order Protocol Distribution', color='blue')

plt.subplot(3, 2, 4)
sns.histplot(df['total_items'], kde=True)
plt.title('Total Items Distribution', color='blue')

plt.subplot(3, 2, 5)
sns.histplot(df['subtotal'], kde=True)
plt.title('Subtotal Distribution', color='blue')

plt.subplot(3, 2, 6)
sns.histplot(df['num_distinct_items'], kde=True)
plt.title('Number of Distinct Items Distribution', color='blue')

plt.tight_layout()
plt.show()
```



In [17]:
```
plt.figure(figsize=(10, 10))

plt.subplot(3, 2, 1)
sns.histplot(df['min_item_price'], kde=True)
plt.title('Minimum Item Price Distribution', color='blue')

plt.subplot(3, 2, 2)
sns.histplot(df['max_item_price'], kde=True)
plt.title('Maximum Item Price Distribution', color='blue')

plt.subplot(3, 2, 3)
sns.histplot(df['total_onshift_dashers'], kde=True)
plt.title('Total Onshift Dashers Distribution', color='blue')

plt.subplot(3, 2, 4)
sns.histplot(df['total_busy_dashers'], kde=True)
```

```
plt.title('Total Busy Dashers Distribution', color='blue')

plt.subplot(3, 2, 5)
sns.histplot(df['total_outstanding_orders'], kde=True)
plt.title('Total Outstanding Orders Distribution', color='blue')

plt.subplot(3, 2, 6)
sns.histplot(df['estimated_store_to_consumer_driving_duration'], kde=True)
plt.title('Estimated Store to Consumer Driving Duration Distribution', color='

plt.tight_layout()
plt.show()
```



Observations:

Skewed Distributions & Outliers: . This suggests that while most

- A large number of numerical features (total_items, subtotal, num_distinct_items, min_item_price, max_item_price, total_onshift_dashers, total_busy_dashers, total_outstanding_orders) exhibit strong right-skewness and a significant presence of outliers.
- This suggests that while most values are clustered at the lower end, there are occasional, much larger values.
- Also estimated store to consumer driving duration feature is more symmterical or uniform in nature.

In [17]:

Scatterplot

In [18]:
```
df_int = df.select_dtypes(include='number')
df_int
```

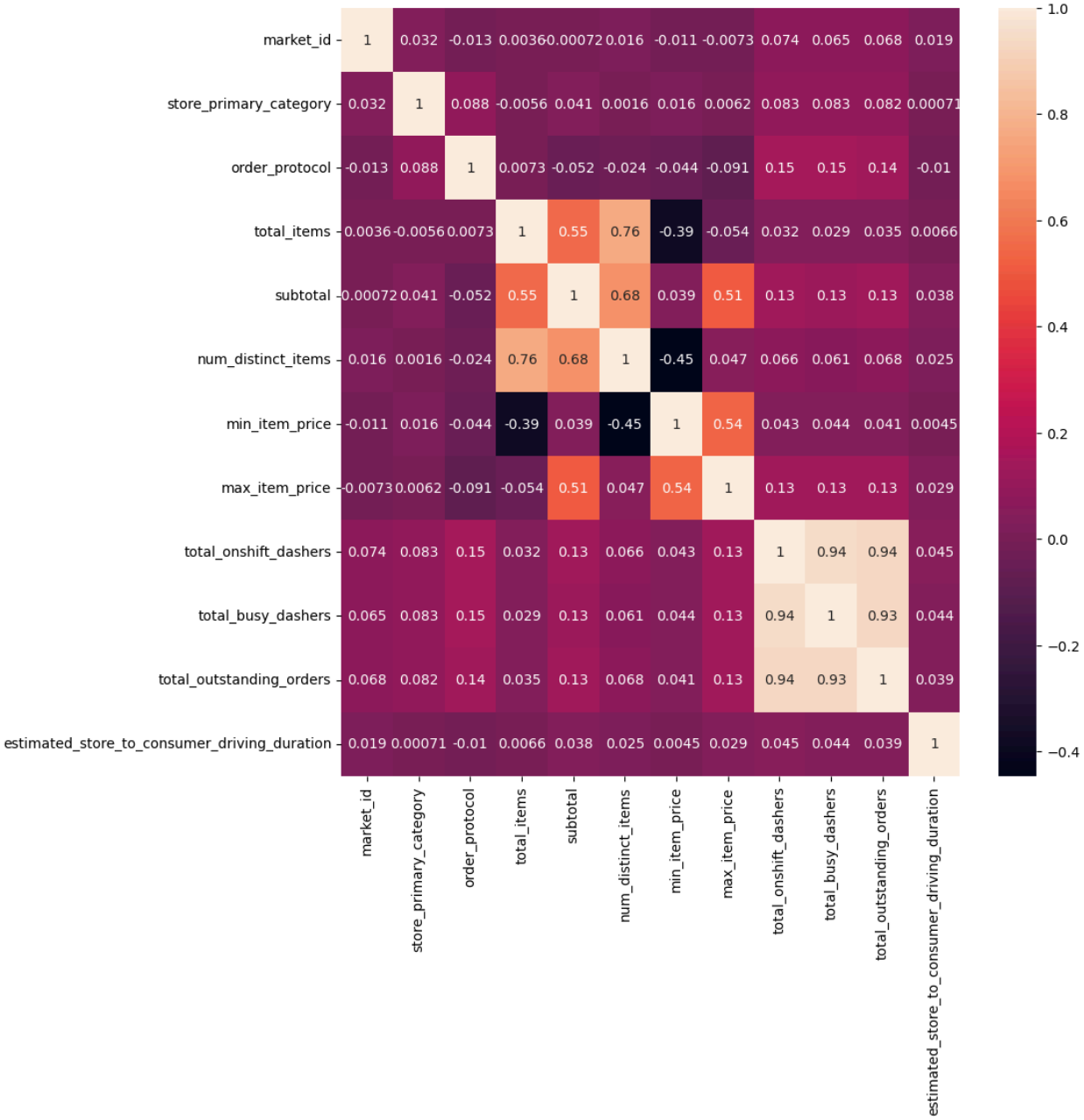
Out[18]:

	market_id	store_primary_category	order_protocol	total_items	subtota
0	1.0	4	1.0	4	344
1	2.0	46	2.0	1	190
2	2.0	36	3.0	4	477
3	1.0	38	1.0	1	152
4	1.0	38	1.0	2	362
...	...	...	...	...	.
175772	1.0	28	4.0	3	138
175773	1.0	28	4.0	6	301
175774	1.0	28	4.0	5	183
175775	1.0	58	1.0	1	117
175776	1.0	58	1.0	4	260

175777 rows × 12 columns

In [19]:
```
plt.figure(figsize=(10, 10))
sns.heatmap(df[['market_id','store_primary_category','order_protocol','total_i
```

Out[19]: <Axes: >



Observations:

- The strong correlations between total_onshift_dashers, total_busy_dashers, and total_outstanding_orders highlight the increased demand (outstanding orders)
- Item column are also postively corelated among themselves.

In [19]:

In [19]:

Skewnes values

Skewness: A positive value indicates a right-skewed distribution (tail to the right, more extreme large values), while a negative value indicates a left-skewed distribution (tail to the left, more extreme small values). A value near zero suggests symmetry.

Kurtosis (Excess Kurtosis): A positive value means the distribution is peaky, with a sharper peak and fatter tails than a normal distribution. A negative value means it's flatter with a broader peak and thinner tails. A value near zero is mesokurtic, similar to a normal distribution.

In [20]:
```
df.select_dtypes(include='number').skew()
```

Out[20]:

	0
market_id	0.225556
store_primary_category	-0.096707
order_protocol	0.109423
total_items	23.286019
subtotal	1.918445
num_distinct_items	1.574312
min_item_price	2.338844
max_item_price	2.204340
total_onshift_dashers	0.856865
total_busy_dashers	0.778571
total_outstanding_orders	1.191167
estimated_store_to_consumer_driving_duration	0.139994

dtype: float64

Observations:

Skewedness value speaks about whether distribution is right tail or left tail. In above most of the numerical features related to order size, price, and dasher availability are highly right-skewed, confirming the presence of significant outliers and a concentration of data at lower values.

In [21]:
```
#market_id        store_primary_category        order_protocol        total_it
```

In [22]:
```
df.select_dtypes(include='number').kurtosis()
```

Out[22]:

	0
market_id	-1.198500
store_primary_category	-1.180777
order_protocol	-1.328975
total_items	3102.967957
subtotal	5.595243
num_distinct_items	4.176660
min_item_price	14.733903
max_item_price	12.962242
total_onshift_dashers	-0.043482
total_busy_dashers	-0.197986
total_outstanding_orders	0.846631
estimated_store_to_consumer_driving_duration	-0.414423

dtype: float64

Observations:

- total_item column is highly peaky and skewed also, whereas price column are less peaky and skewed compare to total_item column
- Dasher columns have flat peaks and are not skewed.

In [22]:
```
```

Feature Engineering

In [23]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   created_at                                    175777 non-null  datetime6
4[ns]
 2   actual_delivery_time                          175777 non-null  datetime6
4[ns]
 3   store_primary_category                        175777 non-null  int64
 4   order_protocol                                175777 non-null  float64
 5   total_items                                   175777 non-null  int64
 6   subtotal                                      175777 non-null  int64
 7   num_distinct_items                            175777 non-null  int64
 8   min_item_price                                175777 non-null  int64
 9   max_item_price                                175777 non-null  int64
 10  total_onshift_dashers                         175777 non-null  float64
 11  total_busy_dashers                            175777 non-null  float64
 12  total_outstanding_orders                      175777 non-null  float64
 13  estimated_store_to_consumer_driving_duration  175777 non-null  float64
dtypes: datetime64[ns](2), float64(6), int64(6)
memory usage: 18.8 MB
```

In [24]:
```
df['created_at'].min(),df['created_at'].max()
```

Out[24]:
```
(Timestamp('2015-01-21 15:22:03'), Timestamp('2015-02-18 06:00:44'))
```

In [25]:
```
#extracting hour ,minuts dayof week, weekend ,weekdays
df['create_hour'] = df['created_at'].dt.hour
df['create_min'] = df['created_at'].dt.minute
df['create_dayofweek'] = df['created_at'].dt.dayofweek
```

In [26]:
```
df['create_min']
```

Out[26]:

	create_min
0	24
1	49
2	11
3	36
4	12
...	...
175772	19
175773	1
175774	46
175775	18
175776	24

175777 rows × 1 columns

dtype: int32

In [27]:
```
df['create_dayofweek'].value_counts()
```

Out[27]:

create_dayofweek	count
5	30858
6	29893
4	25004
0	24202
3	22584
2	21753
1	21483

dtype: int64

In [28]:
```
df['is_weekend'] = df['create_dayofweek'].apply(lambda x: 1 if x>4 else 0)
```

In [29]:
```
df.head(5)
```

Out[29]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_pr
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	

In [30]:
```
df['delivery_time'] = df['actual_delivery_time']-df['created_at']
df['delivery_time'] = df['delivery_time'].apply(lambda x:x.total_seconds()/60)
```

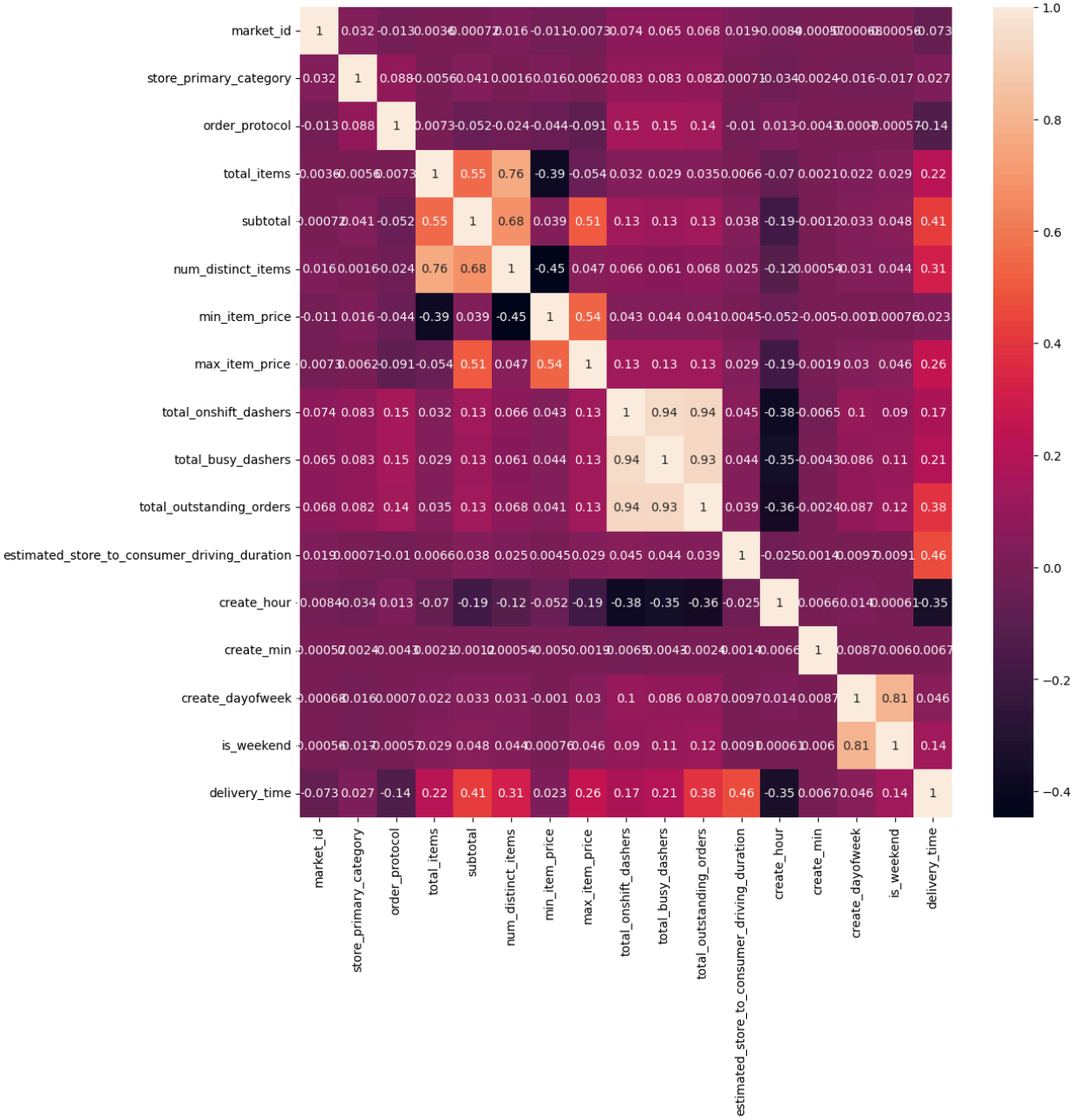
In [31]:
```
df.head(5)
```

Out[31]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_pr
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	

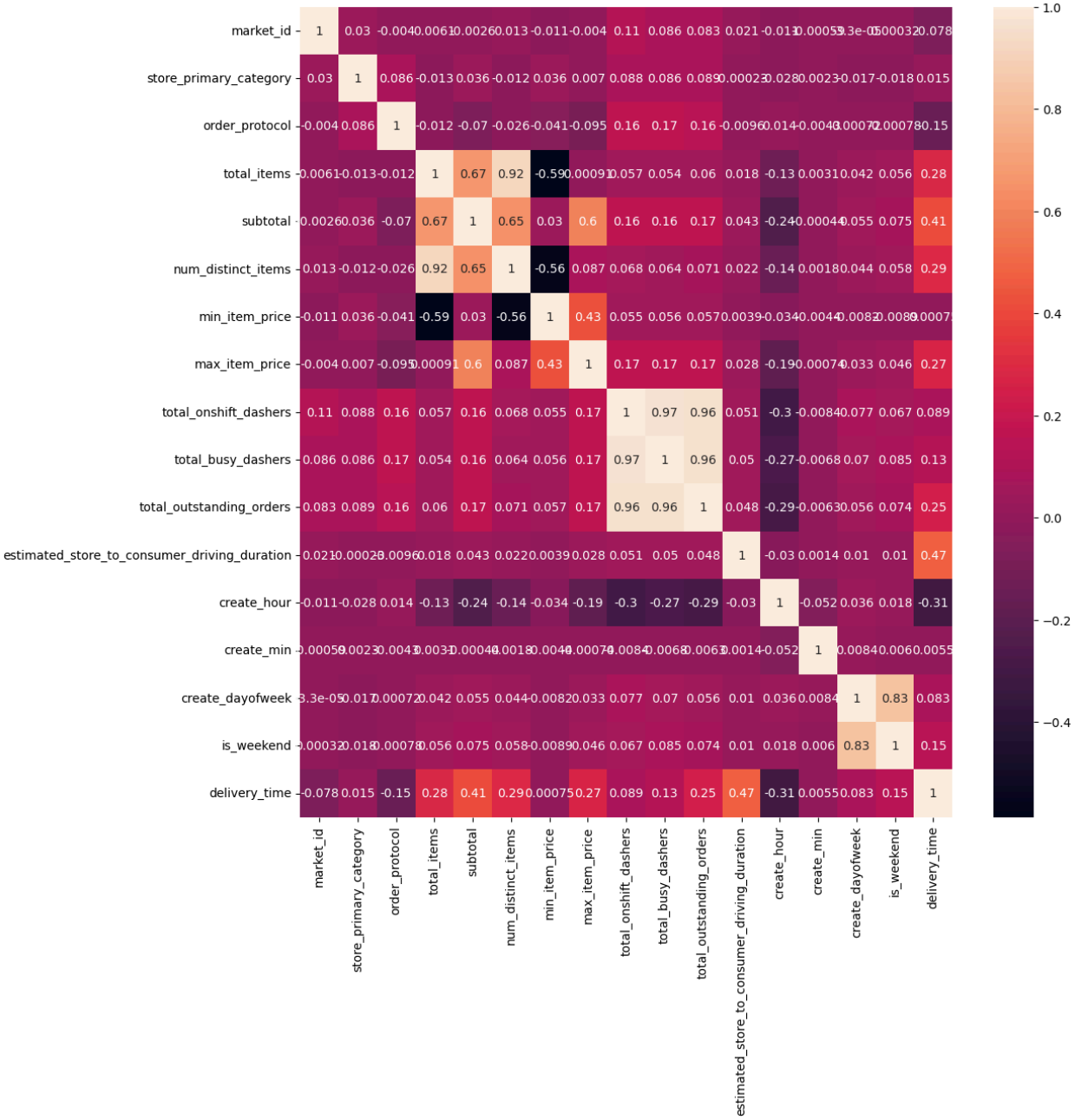
In [32]:
```
#pearson
plt.figure(figsize=(12, 12))
sns.heatmap(df[['market_id','store_primary_category','order_protocol','total_i
```

Out[32]:
```
<Axes: >
```



In [33]:
```
#spearman corelation
plt.figure(figsize=(12, 12))
sns.heatmap(df[['market_id','store_primary_category','order_protocol','total_i
```

Out[33]: <Axes: >



Observations:

- In this we have done feature engineering of the target column i.e delivery time.
- Delivery time and hour are negatively moderately correlated, meaning that delivery time tends to be higher if ordered in the morning (earlier hours).
- Delivery time not highly corelated with other feature but features like estimate_store to consumer driving duration,outstanding order,subtotal are relatively high positively corelated compare to others. Hence a

deeper investigation wil uncover driving factor which will be done by machine learning algo.

Scatterplot

In [34]:
```
df_1 = df.copy()
```

In [35]:
```
sns.pairplot(df_1[['subtotal',        'num_distinct_items',        'min_item_p
```

```
Output hidden; open in https://colab.research.google.com to view.
```

Observations:

- We can see total dasher(onshit and busy) are highly corelated with outstanding order. This is logical, as an increase in outstanding orders often leads to more dashers being busy or on shifts.
- Delivery time are moderately positive corelated with subtotal,estimated_Store_to_consumer_driving_duration,total outstanding order.

In [35]:
```

```

In [35]:
```

```

Outlier Treatment

We saw many outliers in boxplot and hence for predicting target column we have to run algorithm which doesn't have outlier or very less in order to not get biasesd or inaccurate predictions. Outliers can significantly skew model training leading to model that perform poorly on unseen data. So to remove outlier treatment we will use most preferable method called as IQR method.

- Interquartile Range (IQR) method is used to identify and treat outliers. It calculates the IQR as the difference between the 75th (Q3) and 25th (Q1) percentiles. Outliers are then defined as data points falling outside a range of Q1 - 1.5 * IQR and Q3 + 1.5 * IQR. These outliers can then be capped at the calculated lower and upper limits to mitigate their impact on analysis
- Local Outlier Factor

In [35]:

Capping by Interquartile Range (IQR) method

In [36]:
```
# without outlier model
```

In [37]:
```
df_1[['total_items','subtotal',        'num_distinct_items',        'min_item_
```

Out[37]:

	total_items	subtotal	num_distinct_items	min_item_price	max_i
count	175777.000000	175777.000000	175777.000000	175777.000000	1757
mean	3.204976	2697.111147	2.675060	684.965433	11
std	2.674055	1828.554893	1.625681	519.882924	5
min	1.000000	0.000000	1.000000	-86.000000	
25%	2.000000	1412.000000	1.000000	299.000000	7
50%	3.000000	2224.000000	2.000000	595.000000	10
75%	4.000000	3410.000000	3.000000	942.000000	13
max	411.000000	26800.000000	20.000000	14700.000000	147

In [38]:
```
# removing outliers by capping it use IQR method
selected_col = ['total_items','subtotal',        'num_distinct_items',
for col in selected_col:
  Q1 =df_1[col].quantile(0.25)
  Q3 =df_1[col].quantile(0.75)
  IQR = Q3-Q1
  lower_limit = Q1-1.5*IQR
  upper_limit = Q3+1.5*IQR
  df_1[col] = np.where(df_1[col]>upper_limit,upper_limit,np.where(df_1[col]<lo
```

In [39]:
```
for i in df_1.columns:
  print(i , "-->", df_1[i].nunique())
```

```
market_id --> 6
created_at --> 162649
actual_delivery_time --> 160344
store_primary_category --> 73
order_protocol --> 7
total_items --> 7
subtotal --> 5723
num_distinct_items --> 6
min_item_price --> 1752
max_item_price --> 1957
total_onshift_dashers --> 145
total_busy_dashers --> 141
total_outstanding_orders --> 194
estimated_store_to_consumer_driving_duration --> 1176
create_hour --> 19
create_min --> 60
create_dayofweek --> 7
is_weekend --> 2
delivery_time --> 74
```

Capping by Quantile method

In [40]:
```
df_2= df.copy()
```

In [41]:
```
#capping values above 99 percentile in column w
select_col2 = ['total_items','subtotal',        'num_distinct_items',        '
for col in select_col2:
  upper_capping = df_2[col].quantile(.99)
  lower_capping = df_2[col].quantile(.01)
  df_2[col]= np.where(df_2[col]>upper_capping,upper_capping,np.where(df_2[col]
```

In [41]:
```

```

In [41]:
```

```

In [41]:
```

```

Droppping rows above 99 percentile and below 1 percentile

In [42]:
```
df_3= df.copy()
```

In [43]:
```
#extract rows after masking outliers

select_col3 = ['total_items','subtotal',        'num_distinct_items',        '
# Dictionary to store results
outlier_mask = pd.Series(False,index =df_3.index)

for col in select_col3 :
```

```
    upper = df_3[col].quantile(.99)
    lower = df_3[col].quantile(.01)
    outlier_mask |= (df[col] < lower) | (df[col] > upper)

outliers_df = df_3[outlier_mask].copy()

print(outliers_df)
```

```
        market_id          created_at actual_delivery_time  \
12            1.0 2015-02-02 05:27:49  2015-02-02 06:54:49
24            4.0 2015-02-14 03:04:07  2015-02-14 04:11:07
26            3.0 2015-02-12 03:12:08  2015-02-12 03:46:08
29            3.0 2015-02-17 21:34:38  2015-02-17 22:06:38
32            3.0 2015-02-17 01:14:22  2015-02-17 02:17:22
...           ...                 ...                  ...
175680        4.0 2015-01-25 02:42:49  2015-01-25 03:32:49
175684        4.0 2015-02-07 02:13:32  2015-02-07 03:04:32
175690        4.0 2015-01-25 03:13:33  2015-01-25 04:03:33
175731        1.0 2015-02-13 19:38:31  2015-02-13 20:11:31
175761        1.0 2015-02-09 20:32:27  2015-02-09 21:04:27

        store_primary_category  order_protocol  total_items  subtotal  \
12                          38             1.0            7     14900
24                          38             1.0            5      5400
26                           4             5.0            5      6200
29                           4             5.0            2      2641
32                           4             5.0            4      5100
...                        ...             ...          ...       ...
175680                      38             2.0            2      3690
175684                      38             2.0            3      5145
175690                      38             2.0            2      4420
175731                      28             4.0            1       375
175761                      28             4.0            1       437

        num_distinct_items  min_item_price  max_item_price  \
12                       5            1200            3900
24                       3             600            2200
26                       5             500            1800
29                       2             625            1240
32                       4             500            1900
...                    ...             ...             ...
175680                   2            1148            1198
175684                   3            1013            1163
175690                   2            1825            2595
175731                   1             375             375
175761                   1             497             448

        total_onshift_dashers  total_busy_dashers  total_outstanding_orders  \
12                        8.0                11.0                      11.0
24                      130.0               129.0                     230.0
26                       34.0                30.0                      28.0
29                       27.0                22.0                      24.0
32                       16.0                16.0                      26.0
...                       ...                 ...                       ...
175680                  106.0               144.0                     137.0
175684                  137.0                89.0                     159.0
175690                  103.0               147.0                     120.0
175731                   31.0                30.0                      32.0
175761                   36.0                26.0                      27.0

        estimated_store_to_consumer_driving_duration  create_hour  create_min
\
```

```
12                                                 901.0            5          27
24                                                 795.0            3           4
26                                                  86.0            3          12
29                                                  89.0           21          34
32                                                1056.0            1          14
...                                                  ...          ...         ...
175680                                             566.0            2          42
175684                                             569.0            2          13
175690                                             752.0            3          13
175731                                             299.0           19          38
175761                                             314.0           20          32

        create_dayofweek  is_weekend  delivery_time
12                     0           0           87.0
24                     5           1           67.0
26                     3           0           34.0
29                     1           0           32.0
32                     1           0           63.0
...                  ...         ...            ...
175680                 6           1           50.0
175684                 5           1           51.0
175690                 6           1           50.0
175731                 4           0           33.0
175761                 0           0           32.0

[15630 rows x 19 columns]
```

In [44]:
```
outliers_df.index
```

Out[44]:
```
Index([    12,     24,     26,     29,     32,     39,    112,    114,    12
6,
          139,
       ...
       175646, 175655, 175658, 175674, 175678, 175680, 175684, 175690, 17573
1,
       175761],
      dtype='int64', length=15630)
```

In [44]:
```

```

In [44]:
```

```

Dropping rows by Local Outlier Factor method

In [45]:
```
df.head(5)
```

Out[45]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_pr
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	

In [46]:
```
df_4 = df.copy()
```

In [47]:
```
df_4.drop(columns = ['created_at','actual_delivery_time'],inplace = True)
```

In [48]:
```
df_4
```

Out[48]:

	market_id	store_primary_category	order_protocol	total_items	subtota
0	1.0	4	1.0	4	344
1	2.0	46	2.0	1	190
2	2.0	36	3.0	4	477
3	1.0	38	1.0	1	152
4	1.0	38	1.0	2	362
...	...	...	...	...	.
175772	1.0	28	4.0	3	138
175773	1.0	28	4.0	6	301
175774	1.0	28	4.0	5	183
175775	1.0	58	1.0	1	117
175776	1.0	58	1.0	4	260

175777 rows × 17 columns

In [48]:
```
```

In [48]:
```
```

In [49]:
```
from sklearn.decomposition import PCA
from sklearn.neighbors import LocalOutlierFactor
```

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
import numpy as np
df_lof = df.copy()
df_lof.drop(columns = ['created_at','actual_delivery_time'],inplace = True)

X_lof = df_lof.drop('delivery_time', axis=1)
y_lof = df_lof['delivery_time']

categorical_cols_lof = ['market_id', 'store_primary_category', 'order_protocol
numerical_cols_lof = ['total_items', 'subtotal', 'num_distinct_items', 'min_it
                      'total_onshift_dashers', 'total_busy_dashers', 'total_outs
                      'estimated_store_to_consumer_driving_duration', 'is_weeker
preprocessor_lof = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols_lof),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols_lof)
    ])

pipeline_lof = Pipeline(steps=[('preprocessor', preprocessor_lof)])

X_processed_lof = pipeline_lof.fit_transform(X_lof)

X_reduced = PCA(n_components=10).fit_transform(X_processed_lof)
lof = LocalOutlierFactor(n_neighbors=10, contamination='auto', n_jobs=-1)
y_pred_lof = lof.fit_predict(X_reduced)
```

In [50]:
```
X_reduced
```

Out[50]:
```
array([[-0.89001631,  1.24082105,  0.48593554, ..., -0.71623233,
        -0.19933798, -0.20716824],
       [-2.21259957, -1.1541672 ,  1.31398667, ...,  0.76050429,
         0.13344271, -0.07475935],
       [-1.1300154 ,  1.09730235,  1.43530063, ...,  1.27802654,
        -0.31121595, -0.2579107 ],
       ...,
       [-0.42028539,  0.73563651, -1.47477036, ..., -0.3058341 ,
        -0.07432483,  0.16403047],
       [-2.32991671, -0.68928125, -0.73569739, ..., -0.52440625,
        -0.06212664, -0.42817397],
       [-1.1357502 ,  1.19505458, -0.50289932, ..., -0.49010108,
        -0.07438654, -0.53103175]])
```

In [51]:
```
y_pred_lof
```

Out[51]:
```
array([1, 1, 1, ..., 1, 1, 1])
```

In [52]:
```
X_reduced_df = pd.DataFrame(X_reduced)
```

In [53]:
```
y_pred_lof = pd.Series(y_pred_lof)
```

In [54]:
```
Pca_df = pd.concat([X_reduced_df,y_pred_lof],axis=1)
```

In [55]:
```
Pca_df
```

Out[55]:

	0	1	2	3	4	5	6
0	-0.890016	1.240821	0.485936	-0.684094	1.427548	-0.262787	-0.550858
1	-2.212600	-1.154167	1.313987	-0.578961	0.751729	0.473506	-0.329317
2	-1.130015	1.097302	1.435301	-0.726370	-1.109810	-0.202654	0.087321
3	-2.127328	-1.308934	1.648577	-0.568231	1.137372	0.383179	-0.410475
4	-1.626255	-0.242202	2.740957	-0.634850	-1.608526	-0.249546	-0.468655
...	...	...	...	...	...	...	...
175772	-1.764245	0.455345	-1.047910	-0.553593	-0.916404	0.040263	0.005705
175773	-1.368290	1.834096	-0.335893	-0.649980	1.699667	0.548058	-0.025279
175774	-0.420285	0.735637	-1.474770	1.570836	1.149232	0.667237	-0.007278
175775	-2.329917	-0.689281	-0.735697	1.771527	-0.626328	-0.109800	-0.288009
175776	-1.135750	1.195055	-0.502899	1.624867	-1.843979	-0.031332	-0.331576

175777 rows × 11 columns

In [56]:
```
y_pred_lof.value_counts()
```

Out[56]:

	count
1	175430
-1	347

dtype: int64

In [57]:
```
# map the outlier to the df_lof
df_lof['outlier_lof'] = y_pred_lof
```

In [58]:
```
df_lof[df_lof['outlier_lof'] == -1].index
```

Out[58]:
```
Index([   711,    921,   1120,   1645,   1646,   2623,   3313,   3909,   4339,
         5315,
       ...
       170368, 171590, 171632, 172230, 172460, 172619, 172725, 174130, 175178,
       175191],
      dtype='int64', length=347)
```

In [59]:
```
df_lof = df_lof.drop(df_lof[df_lof['outlier_lof'] == -1].index)
df_lof
```

Out[59]:

	market_id	store_primary_category	order_protocol	total_items	subtota
0	1.0	4	1.0	4	344
1	2.0	46	2.0	1	190
2	2.0	36	3.0	4	477
3	1.0	38	1.0	1	152
4	1.0	38	1.0	2	362
...	...	...	...	...	.
175772	1.0	28	4.0	3	138
175773	1.0	28	4.0	6	301
175774	1.0	28	4.0	5	183
175775	1.0	58	1.0	1	117
175776	1.0	58	1.0	4	260

175430 rows × 18 columns

In [60]:
```
df_lof.drop(columns = ['outlier_lof'],inplace = True)
```

In [61]:
```
df_lof
```

Out[61]:

	market_id	store_primary_category	order_protocol	total_items	subtota
0	1.0	4	1.0	4	344
1	2.0	46	2.0	1	190
2	2.0	36	3.0	4	477
3	1.0	38	1.0	1	152
4	1.0	38	1.0	2	362
...	...	...	...	...	.
175772	1.0	28	4.0	3	138
175773	1.0	28	4.0	6	301
175774	1.0	28	4.0	5	183
175775	1.0	58	1.0	1	117
175776	1.0	58	1.0	4	260

175430 rows × 17 columns

In [61]:

In [61]:

In [61]:

Q- Which column to drop from machine learning model?

Ans - We will drop column like 'created_at', 'actual_delivery_time' because these column dont serve in learning the model and also redundant. We already extracted information from these columns (like delivery time)

In [61]:

ML model

Random forest and Xgboost model

1. Using model by IQR Capping method
2. Using model by 99 percentile and 1 percentile Capping method
3. Using model after dropping rows above 99th percentile value and below 1 percentile value

1) Using model by IQR Capping method

In [62]:
```
df_m_1 = df_1.copy()
```

In [63]:
```
columns_to_drop = ['created_at', 'actual_delivery_time']
df_m_1 = df_m_1.drop(columns=columns_to_drop)

display(df_m_1.head())
```

	market_id	store_primary_category	order_protocol	total_items	subtotal	num_
0	1.0	4	1.0	4.0	3441.0	
1	2.0	46	2.0	1.0	1900.0	
2	2.0	36	3.0	4.0	4771.0	
3	1.0	38	1.0	1.0	1525.0	
4	1.0	38	1.0	2.0	3620.0	

In [64]:
```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.ensemble import RandomForestRegressor ,GradientBoostingRegressor
from sklearn.preprocessing import StandardScaler,TargetEncoder ,OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split,GridSearchCV,KFold,cross_
from sklearn.metrics import mean_squared_error,r2_score,mean_absolute_error,me
```

In [65]:
```
x = df_m_1.drop('delivery_time',axis=1)
y = df_m_1['delivery_time']
```

In [66]:
```
labels1 = ['Morning','Afternoon','Evening','Night']
bins1 = [0,6,12,18,24]
x['create_hour']  = pd.cut(x['create_hour'],bins=bins1,labels=labels1, include
```

In [67]:
```
labels3 = ['0-15','15-30','30-45','45-60']
bins3 = [0,15,30,45,60]
x['create_min']  = pd.cut(x['create_min'],bins=bins3,labels=labels3,right = Fa
```

In [68]:
```
day_map = {
    0: 'Monday',
    1: 'Tuesday',
    2: 'Wednesday',
    3: 'Thursday',
    4: 'Friday',
    5: 'Saturday',
    6: 'Sunday'
}

x['create_dayofweek'] = x['create_dayofweek'].map(day_map)
```

In [69]:
```
x
```

Out[69]:

	market_id	store_primary_category	order_protocol	total_items	subtota
0	1.0	4	1.0	4.0	3441.
1	2.0	46	2.0	1.0	1900.
2	2.0	36	3.0	4.0	4771.
3	1.0	38	1.0	1.0	1525.
4	1.0	38	1.0	2.0	3620.
...	...	...	...	...	.
175772	1.0	28	4.0	3.0	1389.
175773	1.0	28	4.0	6.0	3010.
175774	1.0	28	4.0	5.0	1836.
175775	1.0	58	1.0	1.0	1175.
175776	1.0	58	1.0	4.0	2605.

175777 rows × 16 columns

In [70]:
```
y.min(),y.max()
```

Out[70]: (32.0, 110.0)

In [71]:
```
x.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 16 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   store_primary_category                        175777 non-null  int64
 2   order_protocol                                175777 non-null  float64
 3   total_items                                   175777 non-null  float64
 4   subtotal                                      175777 non-null  float64
 5   num_distinct_items                            175777 non-null  float64
 6   min_item_price                                175777 non-null  float64
 7   max_item_price                                175777 non-null  float64
 8   total_onshift_dashers                         175777 non-null  float64
 9   total_busy_dashers                            175777 non-null  float64
 10  total_outstanding_orders                      175777 non-null  float64
 11  estimated_store_to_consumer_driving_duration  175777 non-null  float64
 12  create_hour                                   175777 non-null  category
 13  create_min                                    175777 non-null  category
 14  create_dayofweek                              175777 non-null  object
 15  is_weekend                                    175777 non-null  int64
dtypes: category(2), float64(11), int64(2), object(1)
memory usage: 19.1+ MB
```

Random forest Model

In [72]:
```
#with catboost encoding
```

In [73]:
```
!pip install category_encoders
from sklearn.metrics import make_scorer
from category_encoders import TargetEncoder
from sklearn.model_selection import KFold

from category_encoders import CatBoostEncoder

categorical_cols = x[['market_id', 'store_primary_category', 'order_protocol',
numeric_cols = x[['total_items', 'subtotal', 'num_distinct_items','min_item_pr
boolean_cols = ['is_weekend']

preprocessor = ColumnTransformer(
    transformers=[
        ('cat', CatBoostEncoder(random_state=42), categorical_cols),
        ('num', 'passthrough', numeric_cols),
         ('bool','passthrough',boolean_cols)
    ]
)

pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor(random_state=42))
])

param_dist = { 'model__n_estimators': [10, 20],
               'model__max_depth': [None, 10, 20],
               'model__min_samples_split': [2, 5, 10],
               'model__min_samples_leaf': [1, 2, 10],
               'model__max_features': ['auto', 'sqrt', 'log2'],
               'model__bootstrap': [True],
               'model__criterion': ['squared_error'] }

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, randc

# CV strategy (optional, for RandomizedSearchCV)
cv_strategy = KFold(n_splits=5, shuffle=True, random_state=42)

random_search = RandomizedSearchCV(pipeline,
param_distributions=param_dist,
n_iter=10,
scoring='neg_root_mean_squared_error' ,
refit = True,
cv=cv_strategy,
n_jobs=-1,
random_state=42,
verbose=2 )

random_search.fit(X_train, y_train) #
print("Best Parameters:", random_search.best_params_)
```

```python
print("Best CV RMSE:", -random_search.best_score_)
y_pred = random_search.best_estimator_.predict(X_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_mae = mean_absolute_error(y_test, y_pred)
test_mape = mean_absolute_percentage_error(y_test, y_pred)
test_r2 = r2_score(y_test, y_pred)
accuracy = 1 - test_mape
print("Test RMSE:", test_rmse)
print("Test MAE:", test_mae)
print("Test R^2:", test_r2)
print("Test MAPE:", test_mape)
print("Test Accuracy:", accuracy)
```

```
Collecting category_encoders
  Downloading category_encoders-2.9.0-py3-none-any.whl.metadata (7.9 kB)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.12/dist-
packages (from category_encoders) (2.0.2)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.12/dist-
packages (from category_encoders) (2.2.2)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.12/dist-p
ackages (from category_encoders) (1.0.2)
Requirement already satisfied: scikit-learn>=1.6.0 in /usr/local/lib/python3.1
2/dist-packages (from category_encoders) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.12/dist-p
ackages (from category_encoders) (1.16.3)
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.12/
dist-packages (from category_encoders) (0.14.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python
3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-p
ackages (from pandas>=1.0.5->category_encoders) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis
t-packages (from pandas>=1.0.5->category_encoders) (2025.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn>=1.6.0->category_encoders) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.1
2/dist-packages (from scikit-learn>=1.6.0->category_encoders) (3.6.0)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dis
t-packages (from statsmodels>=0.9.0->category_encoders) (26.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packa
ges (from python-dateutil>=2.8.2->pandas>=1.0.5->category_encoders) (1.17.0)
Downloading category_encoders-2.9.0-py3-none-any.whl (85 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 85.9/85.9 kB 1.7 MB/s eta 0:00:00
Installing collected packages: category_encoders
Successfully installed category_encoders-2.9.0
Fitting 5 folds for each of 10 candidates, totalling 50 fits
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.p
y:528: FitFailedWarning:
10 fits failed out of a total of 50.
The score on these train-test partitions for these parameters will be set to na
n.
If these failures are not expected, you can try to debug them by setting erro
r_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
-
5 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_valida
tion.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1389, in
wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/pipeline.py", line 662,
in fit
    self._final_estimator.fit(Xt, y, **last_step_params["fit"])
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1382, in
wrapper
    estimator._validate_params()
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 436, in
_validate_params
    validate_parameter_constraints(
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/_param_validatio
n.py", line 98, in validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features' param
eter of RandomForestRegressor must be an int in the range [1, inf), a float in
the range (0.0, 1.0], a str among {'log2', 'sqrt'} or None. Got 'auto' instead.

--------------------------------------------------------------------------------
-
5 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_valida
tion.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1389, in
wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/pipeline.py", line 662,
in fit
    self._final_estimator.fit(Xt, y, **last_step_params["fit"])
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1382, in
wrapper
    estimator._validate_params()
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 436, in
```

```
_validate_params
    validate_parameter_constraints(
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/_param_validatio
n.py", line 98, in validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features' param
eter of RandomForestRegressor must be an int in the range [1, inf), a float in
the range (0.0, 1.0], a str among {'sqrt', 'log2'} or None. Got 'auto' instead.

  warnings.warn(some_fits_failed_message, FitFailedWarning)
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:110
8: UserWarning: One or more of the test scores are non-finite: [-3.48324784
nan -3.09810521         nan -4.54076763 -3.06245038
 -4.48480747 -3.3731768  -4.5503311  -3.26821552]
  warnings.warn(
Best Parameters: {'model__n_estimators': 20, 'model__min_samples_split': 10, 'm
odel__min_samples_leaf': 2, 'model__max_features': 'sqrt', 'model__max_depth':
None, 'model__criterion': 'squared_error', 'model__bootstrap': True}
Best CV RMSE: 3.0624503790620587
Test RMSE: 2.9633571127981337
Test MAE: 2.1716694277805857
Test R^2: 0.8996714513581732
Test MAPE: 0.04682113208207755
Test Accuracy: 0.9531788679179225
```

In [73]:

In [73]:

In [73]:

In [73]:

XGboost Model

In [74]:

```
!pip install category_encoders
!pip install xgboost
import xgboost as xgb
from xgboost import XGBRegressor

categorical_cols = x[['market_id', 'store_primary_category', 'order_protocol',
numeric_cols = x[['total_items', 'subtotal', 'num_distinct_items','min_item_pr
boolean_cols = ['is_weekend']
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', CatBoostEncoder(random_state=42), categorical_cols),
        ('num', 'passthrough', numeric_cols),
         ('bool','passthrough',boolean_cols)
    ]
)

pipeline = Pipeline(steps=[
```

```
    ('preprocessor', preprocessor),
    ('model', xgb.XGBRegressor(objective ='reg:squarederror',random_state=42,r
])

param_dist = { 'model__n_estimators': [20, 30],
               'model__max_depth': [3, 5, 7],
               'model__learning_rate': [0.01, 0.1, 0.2],
               'model__subsample': [0.8, 1.0],
               'model__colsample_bytree': [0.8, 1.0],
               'model__min_child_weight': [1, 5, 10]
                }

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, randc
# CV strategy (optional, for RandomizedSearchCV)
cv_strategy = KFold(n_splits=5, shuffle=True, random_state=42)

random_search = RandomizedSearchCV(pipeline,
param_distributions=param_dist,
n_iter=10,
scoring='neg_root_mean_squared_error' ,
refit = True,
cv= cv_strategy,
n_jobs=-1,
random_state=42,
verbose=2 )

random_search.fit(X_train, y_train) #
print("Best Parameters:", random_search.best_params_)
print("Best CV RMSE:", -random_search.best_score_)
y_pred = random_search.best_estimator_.predict(X_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_mae = mean_absolute_error(y_test, y_pred)
test_mape = mean_absolute_percentage_error(y_test, y_pred)
test_r2 = r2_score(y_test, y_pred)
accuracy = 1 - test_mape
print("Test RMSE:", test_rmse)
print("Test MAE:", test_mae)
print("Test R^2:", test_r2)
print("Test MAPE:", test_mape)
print("Test Accuracy:", accuracy)
```

```
Requirement already satisfied: category_encoders in /usr/local/lib/python3.12/dist-packages (2.9.0)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (2.0.2)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (2.2.2)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (1.0.2)
Requirement already satisfied: scikit-learn>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (1.16.3)
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from category_encoders) (0.14.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2025.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.6.0->category_encoders) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=1.6.0->category_encoders) (3.6.0)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dist-packages (from statsmodels>=0.9.0->category_encoders) (26.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0.5->category_encoders) (1.17.0)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.7)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best Parameters: {'model__subsample': 1.0, 'model__n_estimators': 20, 'model__min_child_weight': 1, 'model__max_depth': 7, 'model__learning_rate': 0.2, 'model__colsample_bytree': 1.0}
Best CV RMSE: 2.543682942308041
Test RMSE: 2.591053461585093
Test MAE: 1.936396582051476
Test R^2: 0.9232975396001919
Test MAPE: 0.041756411329210485
Test Accuracy: 0.9582435886707895
```

In [74]:

In [74]:

2) Using model by 99 percentile and 1 percentile Capping method

In [75]:
```
df_m_2 = df_2.copy()
```

In [76]:
```
columns_to_drop = ['created_at', 'actual_delivery_time']
df_m_2 = df_m_2.drop(columns=columns_to_drop)

display(df_m_2.head())
```

	market_id	store_primary_category	order_protocol	total_items	subtotal	num_
0	1.0	4	1.0	4.0	3441.0	
1	2.0	46	2.0	1.0	1900.0	
2	2.0	36	3.0	4.0	4771.0	
3	1.0	38	1.0	1.0	1525.0	
4	1.0	38	1.0	2.0	3620.0	

In [77]:
```
x = df_m_2.drop('delivery_time',axis=1)
y = df_m_2['delivery_time']
```

In [78]:
```
labels1 = ['Morning','Afternoon','Evening','Night']
bins1 = [0,6,12,18,24]
x['create_hour']  = pd.cut(x['create_hour'],bins=bins1,labels=labels1, include
```

In [79]:
```
labels3 = ['0-15','15-30','30-45','45-60']
bins3 = [0,15,30,45,60]
x['create_min']  = pd.cut(x['create_min'],bins=bins3,labels=labels3,right = Fa
```

In [80]:
```
day_map = {
    0: 'Monday',
    1: 'Tuesday',
    2: 'Wednesday',
    3: 'Thursday',
    4: 'Friday',
    5: 'Saturday',
    6: 'Sunday'
}

x['create_dayofweek'] = x['create_dayofweek'].map(day_map)
```

Random Forest model

In [81]:
```
categorical_cols = x[['market_id', 'store_primary_category', 'order_protocol',
numeric_cols = x[['total_items', 'subtotal', 'num_distinct_items','min_item_pr
boolean_cols = ['is_weekend']
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', CatBoostEncoder(random_state=42), categorical_cols),
        ('num', 'passthrough', numeric_cols),
         ('bool','passthrough',boolean_cols)
    ]
)

pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor(random_state=42))
])

param_dist = { 'model__n_estimators': [10, 20],
               'model__max_depth': [None, 10, 20],
               'model__min_samples_split': [2, 5, 10],
               'model__min_samples_leaf': [1, 2, 10],
               'model__max_features': ['auto', 'sqrt', 'log2'],
               'model__bootstrap': [True],
               'model__criterion': ['squared_error'] }

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, randc
# CV strategy (optional, for RandomizedSearchCV)
cv_strategy = KFold(n_splits=5, shuffle=True, random_state=42)

random_search = RandomizedSearchCV(pipeline,
param_distributions=param_dist,
n_iter=10,
scoring='neg_root_mean_squared_error' ,
refit = True,
cv= cv_strategy,
n_jobs=-1,
random_state=42,
verbose=2 )

random_search.fit(X_train, y_train) #
print("Best Parameters:", random_search.best_params_)
print("Best CV RMSE:", -random_search.best_score_)
y_pred = random_search.best_estimator_.predict(X_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_mae = mean_absolute_error(y_test, y_pred)
test_mape = mean_absolute_percentage_error(y_test, y_pred)
test_r2 = r2_score(y_test, y_pred)
accuracy = 1 - test_mape
print("Test RMSE:", test_rmse)
print("Test MAE:", test_mae)
print("Test R^2:", test_r2)
print("Test MAPE:", test_mape)
print("Test Accuracy:", accuracy)
```

```
Fitting 5 folds for each of 10 candidates, totalling 50 fits
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.p
y:528: FitFailedWarning:
10 fits failed out of a total of 50.
The score on these train-test partitions for these parameters will be set to na
n.
If these failures are not expected, you can try to debug them by setting erro
r_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
-
5 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_valida
tion.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1389, in
wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/pipeline.py", line 662,
in fit
    self._final_estimator.fit(Xt, y, **last_step_params["fit"])
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1382, in
wrapper
    estimator._validate_params()
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 436, in
_validate_params
    validate_parameter_constraints(
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/_param_validatio
n.py", line 98, in validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features' param
eter of RandomForestRegressor must be an int in the range [1, inf), a float in
the range (0.0, 1.0], a str among {'log2', 'sqrt'} or None. Got 'auto' instead.

--------------------------------------------------------------------------------
-
5 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_valida
tion.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1389, in
wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/pipeline.py", line 662,
in fit
    self._final_estimator.fit(Xt, y, **last_step_params["fit"])
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1382, in
wrapper
    estimator._validate_params()
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 436, in
```

```
_validate_params
    validate_parameter_constraints(
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/_param_validatio
n.py", line 98, in validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features' param
eter of RandomForestRegressor must be an int in the range [1, inf), a float in
the range (0.0, 1.0], a str among {'sqrt', 'log2'} or None. Got 'auto' instead.

  warnings.warn(some_fits_failed_message, FitFailedWarning)
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:110
8: UserWarning: One or more of the test scores are non-finite: [-3.42021299
nan -3.03373801         nan -4.51554643 -3.00851765
 -4.40834935 -3.30824128 -4.4843865  -3.18263919]
  warnings.warn(
Best Parameters: {'model__n_estimators': 20, 'model__min_samples_split': 10, 'm
odel__min_samples_leaf': 2, 'model__max_features': 'sqrt', 'model__max_depth':
None, 'model__criterion': 'squared_error', 'model__bootstrap': True}
Best CV RMSE: 3.0085176493392987
Test RMSE: 2.8820035969019253
Test MAE: 2.120128337216288
Test R^2: 0.9051045074667028
Test MAPE: 0.04591819029257357
Test Accuracy: 0.9540818097074264
```

XGboost Model

In [82]:

```
!pip install category_encoders
!pip install xgboost
import xgboost as xgb
from xgboost import XGBRegressor

from sklearn.metrics import make_scorer
from category_encoders import TargetEncoder

categorical_cols = x[['market_id', 'store_primary_category', 'order_protocol',
numeric_cols = x[['total_items', 'subtotal', 'num_distinct_items','min_item_pr
boolean_cols = ['is_weekend']
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', CatBoostEncoder(random_state=42), categorical_cols),
        ('num', 'passthrough', numeric_cols),
         ('bool','passthrough',boolean_cols)
    ]
)

pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', xgb.XGBRegressor(objective ='reg:squarederror',random_state=42,r
])

param_dist = { 'model__n_estimators': [20, 30],
               'model__max_depth': [3, 5, 7],
```

```
                'model__learning_rate': [0.01, 0.1, 0.2],
                'model__subsample': [0.8, 1.0],
                'model__colsample_bytree': [0.8, 1.0],
                'model__min_child_weight': [1, 5, 10]
                 }

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, randc
# CV strategy (optional, for RandomizedSearchCV)
cv_strategy = KFold(n_splits=5, shuffle=True, random_state=42)

random_search = RandomizedSearchCV(pipeline,
param_distributions=param_dist,
n_iter=10,
scoring='neg_root_mean_squared_error' ,
refit = True,
cv= cv_strategy,
n_jobs=-1,
random_state=42,
verbose=2 )

random_search.fit(X_train, y_train) #
print("Best Parameters:", random_search.best_params_)
print("Best CV RMSE:", -random_search.best_score_)
y_pred = random_search.best_estimator_.predict(X_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_mae = mean_absolute_error(y_test, y_pred)
test_mape = mean_absolute_percentage_error(y_test, y_pred)
test_r2 = r2_score(y_test, y_pred)
accuracy = 1 - test_mape
print("Test RMSE:", test_rmse)
print("Test MAE:", test_mae)
print("Test R^2:", test_r2)
print("Test MAPE:", test_mape)
print("Test Accuracy:", accuracy)
```

```
Requirement already satisfied: category_encoders in /usr/local/lib/python3.12/d
ist-packages (2.9.0)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.12/dist-
packages (from category_encoders) (2.0.2)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.12/dist-
packages (from category_encoders) (2.2.2)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.12/dist-p
ackages (from category_encoders) (1.0.2)
Requirement already satisfied: scikit-learn>=1.6.0 in /usr/local/lib/python3.1
2/dist-packages (from category_encoders) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.12/dist-p
ackages (from category_encoders) (1.16.3)
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.12/
dist-packages (from category_encoders) (0.14.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python
3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-p
ackages (from pandas>=1.0.5->category_encoders) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis
t-packages (from pandas>=1.0.5->category_encoders) (2025.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn>=1.6.0->category_encoders) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.1
2/dist-packages (from scikit-learn>=1.6.0->category_encoders) (3.6.0)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dis
t-packages (from statsmodels>=0.9.0->category_encoders) (26.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packa
ges (from python-dateutil>=2.8.2->pandas>=1.0.5->category_encoders) (1.17.0)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packag
es (3.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/di
st-packages (from xgboost) (2.29.7)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages
(from xgboost) (1.16.3)
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best Parameters: {'model__subsample': 1.0, 'model__n_estimators': 20, 'model__m
in_child_weight': 1, 'model__max_depth': 7, 'model__learning_rate': 0.2, 'mode
l__colsample_bytree': 1.0}
Best CV RMSE: 2.4746406761844364
Test RMSE: 2.492889383321711
Test MAE: 1.8620150968447686
Test R^2: 0.9289993109801532
Test MAPE: 0.04018453595774417
Test Accuracy: 0.9598154640422558
```

In [82]:

3) Using model after dropping rows above 99th percentile value and below 1 percentile value

In [83]:
```
#dropping outliers_df rows from main
df_m_3 = df.drop(outliers_df.index)
```

In [84]:
```
columns_to_drop = ['created_at', 'actual_delivery_time']
df_m_3 = df_m_3.drop(columns=columns_to_drop)

display(df_m_3.head())
```

	market_id	store_primary_category	order_protocol	total_items	subtotal	num_
0	1.0	4	1.0	4	3441	
1	2.0	46	2.0	1	1900	
2	2.0	36	3.0	4	4771	
3	1.0	38	1.0	1	1525	
4	1.0	38	1.0	2	3620	

In [85]:
```
x = df_m_3.drop('delivery_time',axis=1)
y = df_m_3['delivery_time']
```

In [86]:
```
labels1 = ['Morning','Afternoon','Evening','Night']
bins1 = [0,6,12,18,24]
x['create_hour']  = pd.cut(x['create_hour'],bins=bins1,labels=labels1, include
```

In [87]:
```
labels3 = ['0-15','15-30','30-45','45-60']
bins3 = [0,15,30,45,60]
x['create_min']  = pd.cut(x['create_min'],bins=bins3,labels=labels3,right = Fa
```

In [88]:
```
day_map = {
    0: 'Monday',
    1: 'Tuesday',
    2: 'Wednesday',
    3: 'Thursday',
    4: 'Friday',
    5: 'Saturday',
    6: 'Sunday'
}

x['create_dayofweek'] = x['create_dayofweek'].map(day_map)
```

Random Forest model

In [89]:
```
!pip install category_encoders
```

```
from sklearn.metrics import make_scorer
from category_encoders import TargetEncoder

categorical_cols = x[['market_id', 'store_primary_category', 'order_protocol',
numeric_cols = x[['total_items', 'subtotal', 'num_distinct_items','min_item_pr
boolean_cols = ['is_weekend']
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', CatBoostEncoder(random_state=42), categorical_cols),
        ('num', 'passthrough', numeric_cols),
         ('bool','passthrough',boolean_cols)
    ]
)

pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor(random_state=42))
])

param_dist = { 'model__n_estimators': [10, 20],
               'model__max_depth': [None, 10, 20],
               'model__min_samples_split': [2, 5, 10],
               'model__min_samples_leaf': [1, 2, 10],
               'model__max_features': ['auto', 'sqrt', 'log2'],
               'model__bootstrap': [True],
               'model__criterion': ['squared_error'] }

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, randc
# CV strategy (optional, for RandomizedSearchCV)
cv_strategy = KFold(n_splits=5, shuffle=True, random_state=42)
random_search = RandomizedSearchCV(pipeline,
param_distributions=param_dist,
n_iter=10,
scoring='neg_root_mean_squared_error' ,
refit = True,
cv= cv_strategy,
n_jobs=-1,
random_state=42,
verbose=2 )

random_search.fit(X_train, y_train) #
print("Best Parameters:", random_search.best_params_)
print("Best CV RMSE:", -random_search.best_score_)
y_pred = random_search.best_estimator_.predict(X_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_mae = mean_absolute_error(y_test, y_pred)
test_mape = mean_absolute_percentage_error(y_test, y_pred)
test_r2 = r2_score(y_test, y_pred)
accuracy = 1 - test_mape
print("Test RMSE:", test_rmse)
print("Test MAE:", test_mae)
print("Test R^2:", test_r2)
print("Test MAPE:", test_mape)
```

```
print("Test Accuracy:", accuracy)
```

```
Requirement already satisfied: category_encoders in /usr/local/lib/python3.12/d
ist-packages (2.9.0)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.12/dist-
packages (from category_encoders) (2.0.2)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.12/dist-
packages (from category_encoders) (2.2.2)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.12/dist-p
ackages (from category_encoders) (1.0.2)
Requirement already satisfied: scikit-learn>=1.6.0 in /usr/local/lib/python3.1
2/dist-packages (from category_encoders) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.12/dist-p
ackages (from category_encoders) (1.16.3)
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.12/
dist-packages (from category_encoders) (0.14.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python
3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-p
ackages (from pandas>=1.0.5->category_encoders) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis
t-packages (from pandas>=1.0.5->category_encoders) (2025.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn>=1.6.0->category_encoders) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.1
2/dist-packages (from scikit-learn>=1.6.0->category_encoders) (3.6.0)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dis
t-packages (from statsmodels>=0.9.0->category_encoders) (26.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packa
ges (from python-dateutil>=2.8.2->pandas>=1.0.5->category_encoders) (1.17.0)
Fitting 5 folds for each of 10 candidates, totalling 50 fits
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.p
y:528: FitFailedWarning:
10 fits failed out of a total of 50.
The score on these train-test partitions for these parameters will be set to na
n.
If these failures are not expected, you can try to debug them by setting erro
r_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
-
5 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_valida
tion.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1389, in
wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/pipeline.py", line 662,
in fit
    self._final_estimator.fit(Xt, y, **last_step_params["fit"])
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1382, in
wrapper
    estimator._validate_params()
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 436, in
_validate_params
    validate_parameter_constraints(
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/_param_validatio
n.py", line 98, in validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features' param
eter of RandomForestRegressor must be an int in the range [1, inf), a float in
the range (0.0, 1.0], a str among {'log2', 'sqrt'} or None. Got 'auto' instead.

--------------------------------------------------------------------------------
-
5 fits failed with the following error:
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_valida
tion.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1389, in
wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/sklearn/pipeline.py", line 662,
in fit
    self._final_estimator.fit(Xt, y, **last_step_params["fit"])
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 1382, in
wrapper
    estimator._validate_params()
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line 436, in
```

```
_validate_params
    validate_parameter_constraints(
  File "/usr/local/lib/python3.12/dist-packages/sklearn/utils/_param_validatio
n.py", line 98, in validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features' param
eter of RandomForestRegressor must be an int in the range [1, inf), a float in
the range (0.0, 1.0], a str among {'sqrt', 'log2'} or None. Got 'auto' instead.

  warnings.warn(some_fits_failed_message, FitFailedWarning)
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:110
8: UserWarning: One or more of the test scores are non-finite: [-3.35010488
nan -2.91501142         nan -4.37187652 -2.9123803
 -4.27746583 -3.21281457 -4.3567905  -3.01965117]
  warnings.warn(
Best Parameters: {'model__n_estimators': 20, 'model__min_samples_split': 10, 'm
odel__min_samples_leaf': 2, 'model__max_features': 'sqrt', 'model__max_depth':
None, 'model__criterion': 'squared_error', 'model__bootstrap': True}
Best CV RMSE: 2.912380302275578
Test RMSE: 2.8056155815942745
Test MAE: 2.075180334989652
Test R^2: 0.9008348468676269
Test MAPE: 0.045675319495680274
Test Accuracy: 0.9543246805043197
```

XGboost Model

In [90]:

```
!pip install category_encoders
!pip install xgboost
import xgboost as xgb
from xgboost import XGBRegressor

from sklearn.metrics import make_scorer
from category_encoders import TargetEncoder

# Redefine x and y for this specific model (df_m_3)
x = df_m_3.drop('delivery_time',axis=1)
y = df_m_3['delivery_time']

# Reapply feature engineering to the current x
labels1 = ['Morning','Afternoon','Evening','Night']
bins1 = [0,6,12,18,24]
x['create_hour']  = pd.cut(x['create_hour'],bins=bins1,labels=labels1, include

labels3 = ['0-15','15-30','30-45','45-60']
bins3 = [0,15,30,45,60]
x['create_min']  = pd.cut(x['create_min'],bins=bins3,labels=labels3,right = Fa

day_map = {
    0: 'Monday',
    1: 'Tuesday',
    2: 'Wednesday',
    3: 'Thursday',
```

```
    4: 'Friday',
    5: 'Saturday',
    6: 'Sunday'
}
x['create_dayofweek'] = x['create_dayofweek'].map(day_map)

categorical_cols = x[['market_id', 'store_primary_category', 'order_protocol',
numeric_cols = x[['total_items', 'subtotal', 'num_distinct_items','min_item_pr
boolean_cols = ['is_weekend']
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', CatBoostEncoder(random_state=42), categorical_cols),
        ('num', 'passthrough', numeric_cols),
         ('bool','passthrough',boolean_cols)
    ]
)

pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', xgb.XGBRegressor(objective ='reg:squarederror',random_state=42,r
])

param_dist = { 'model__n_estimators': [20, 30],
               'model__max_depth': [3, 5, 7],
               'model__learning_rate': [0.01, 0.1, 0.2],
               'model__subsample': [0.8, 1.0],
               'model__colsample_bytree': [0.8, 1.0],
               'model__min_child_weight': [1, 5, 10]
                }

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, randc
# CV strategy (optional, for RandomizedSearchCV)
cv_strategy = KFold(n_splits=5, shuffle=True, random_state=42)

random_search = RandomizedSearchCV(pipeline,
param_distributions=param_dist,
n_iter=10,
scoring='neg_root_mean_squared_error' ,
refit = True,
cv= cv_strategy,
n_jobs=-1,
random_state=42,
verbose=2 )

random_search.fit(X_train, y_train) #
print("Best Parameters:", random_search.best_params_)
print("Best CV RMSE:", -random_search.best_score_)
y_pred = random_search.best_estimator_.predict(X_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_mae = mean_absolute_error(y_test, y_pred)
test_mape = mean_absolute_percentage_error(y_test, y_pred)
test_r2 = r2_score(y_test, y_pred)
accuracy = 1 - test_mape
```

```
print("Test RMSE:", test_rmse)
print("Test MAE:", test_mae)
print("Test R^2:", test_r2)
print("Test MAPE:", test_mape)
print("Test Accuracy:", accuracy)
```

```
Requirement already satisfied: category_encoders in /usr/local/lib/python3.12/d
ist-packages (2.9.0)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.12/dist-
packages (from category_encoders) (2.0.2)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.12/dist-
packages (from category_encoders) (2.2.2)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.12/dist-p
ackages (from category_encoders) (1.0.2)
Requirement already satisfied: scikit-learn>=1.6.0 in /usr/local/lib/python3.1
2/dist-packages (from category_encoders) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.12/dist-p
ackages (from category_encoders) (1.16.3)
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.12/
dist-packages (from category_encoders) (0.14.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python
3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-p
ackages (from pandas>=1.0.5->category_encoders) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis
t-packages (from pandas>=1.0.5->category_encoders) (2025.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn>=1.6.0->category_encoders) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.1
2/dist-packages (from scikit-learn>=1.6.0->category_encoders) (3.6.0)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dis
t-packages (from statsmodels>=0.9.0->category_encoders) (26.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packa
ges (from python-dateutil>=2.8.2->pandas>=1.0.5->category_encoders) (1.17.0)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packag
es (3.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/di
st-packages (from xgboost) (2.29.7)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages
(from xgboost) (1.16.3)
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best Parameters: {'model__subsample': 1.0, 'model__n_estimators': 20, 'model__m
in_child_weight': 10, 'model__max_depth': 7, 'model__learning_rate': 0.2, 'mode
l__colsample_bytree': 1.0}
Best CV RMSE: 2.341083530692174
Test RMSE: 2.349545263672575
Test MAE: 1.7726464629731549
Test R^2: 0.9304542829399154
Test MAPE: 0.038998336749447145
Test Accuracy: 0.9610016632505528
```

In [90]:

In [90]:

3) Using model after dropping rows by LOF method

XGboost model

In [91]:
```
df_lof
```

Out[91]:

	market_id	store_primary_category	order_protocol	total_items	subtota
0	1.0	4	1.0	4	344
1	2.0	46	2.0	1	190
2	2.0	36	3.0	4	477
3	1.0	38	1.0	1	152
4	1.0	38	1.0	2	362
...	...	...	...	...	.
175772	1.0	28	4.0	3	138
175773	1.0	28	4.0	6	301
175774	1.0	28	4.0	5	183
175775	1.0	58	1.0	1	117
175776	1.0	58	1.0	4	260

175430 rows × 17 columns

In [91]:

In [91]:

In [92]:
```
!pip install category_encoders
!pip install xgboost

import numpy as np
import pandas as pd
import xgboost as xgb
from xgboost import XGBRegressor

from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.metrics import mean_squared_error, mean_absolute_error, mean_absc
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from category_encoders import TargetEncoder
```

```
x_lof = df_lof.drop('delivery_time', axis=1)
y_lof = df_lof['delivery_time']

x_lof['create_hour'] = pd.to_numeric(x_lof['create_hour'], errors='coerce')
x_lof['create_min'] = pd.to_numeric(x_lof['create_min'], errors='coerce')

labels1 = ['Morning','Afternoon','Evening','Night']
bins1 = [0,6,12,18,24]
x_lof['create_hour'] = pd.cut(x_lof['create_hour'], bins=bins1, labels=labels1


labels3 = ['0-15','15-30','30-45','45-60']
bins3 = [0,15,30,45,60]
x_lof['create_min'] = pd.cut(x_lof['create_min'], bins=bins3, labels=labels3,

day_map = {
    0: 'Monday', 1: 'Tuesday', 2: 'Wednesday',
    3: 'Thursday', 4: 'Friday', 5: 'Saturday', 6: 'Sunday'
}
x_lof['create_dayofweek'] = x_lof['create_dayofweek'].map(day_map)


categorical_cols = ['market_id', 'store_primary_category', 'order_protocol',
                    'create_hour','create_min','create_dayofweek']
numeric_cols = ['total_items', 'subtotal', 'num_distinct_items',
                'min_item_price','max_item_price','total_onshift_dashers',
                'total_busy_dashers','total_outstanding_orders',
                'estimated_store_to_consumer_driving_duration']
boolean_cols = ['is_weekend']

for col in categorical_cols:
    x_lof[col] = x_lof[col].astype(str).fillna("Unknown")

preprocessor = ColumnTransformer(
    transformers=[
        ('cat', CatBoostEncoder(random_state=42), categorical_cols),
        ('num', 'passthrough', numeric_cols),
        ('bool', 'passthrough', boolean_cols)
    ]
)

pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', XGBRegressor(
        objective='reg:squarederror',
        random_state=42,
        n_jobs=-1
    ))
])


param_dist = {
```

```
    'model__n_estimators': [20, 30],
    'model__max_depth': [3, 5, 7],
    'model__learning_rate': [0.01, 0.1, 0.2],
    'model__subsample': [0.8, 1.0],
    'model__colsample_bytree': [0.8, 1.0],
    'model__min_child_weight': [1, 5, 10]
}

X_train, X_test, y_train, y_test = train_test_split(
    x_lof, y_lof, test_size=0.2, random_state=42
)
# CV strategy (optional, for RandomizedSearchCV)
cv_strategy = KFold(n_splits=5, shuffle=True, random_state=42)

random_search = RandomizedSearchCV(
    pipeline,
    param_distributions=param_dist,
    n_iter=10,
    scoring='neg_root_mean_squared_error',
    refit=True,
    cv= cv_strategy,
    n_jobs=-1,
    random_state=42,
    verbose=2
)

random_search.fit(X_train, y_train)

print("Best Parameters:", random_search.best_params_)
print("Best CV RMSE:", -random_search.best_score_)

y_pred = random_search.best_estimator_.predict(X_test)

test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_mae = mean_absolute_error(y_test, y_pred)
test_mape = mean_absolute_percentage_error(y_test, y_pred)
test_r2 = r2_score(y_test, y_pred)
accuracy = 1 - test_mape

print("Test RMSE:", test_rmse)
print("Test MAE:", test_mae)
print("Test R^2:", test_r2)
print("Test MAPE:", test_mape)
print("Test Accuracy:", accuracy)
```

```
Requirement already satisfied: category_encoders in /usr/local/lib/python3.12/d
ist-packages (2.9.0)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.12/dist-
packages (from category_encoders) (2.0.2)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.12/dist-
packages (from category_encoders) (2.2.2)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.12/dist-p
ackages (from category_encoders) (1.0.2)
Requirement already satisfied: scikit-learn>=1.6.0 in /usr/local/lib/python3.1
2/dist-packages (from category_encoders) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.12/dist-p
ackages (from category_encoders) (1.16.3)
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.12/
dist-packages (from category_encoders) (0.14.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python
3.12/dist-packages (from pandas>=1.0.5->category_encoders) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-p
ackages (from pandas>=1.0.5->category_encoders) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis
t-packages (from pandas>=1.0.5->category_encoders) (2025.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn>=1.6.0->category_encoders) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.1
2/dist-packages (from scikit-learn>=1.6.0->category_encoders) (3.6.0)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dis
t-packages (from statsmodels>=0.9.0->category_encoders) (26.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packa
ges (from python-dateutil>=2.8.2->pandas>=1.0.5->category_encoders) (1.17.0)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packag
es (3.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/di
st-packages (from xgboost) (2.29.7)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages
(from xgboost) (1.16.3)
Fitting 5 folds for each of 10 candidates, totalling 50 fits
Best Parameters: {'model__subsample': 1.0, 'model__n_estimators': 20, 'model__m
in_child_weight': 1, 'model__max_depth': 7, 'model__learning_rate': 0.2, 'mode
l__colsample_bytree': 1.0}
Best CV RMSE: 2.4695278453085683
Test RMSE: 2.471033471717564
Test MAE: 1.8734184392996378
Test R^2: 0.930033861542739
Test MAPE: 0.040914202952876705
Test Accuracy: 0.9590857970471233
```

In [92]:

In [92]:

In [92]:

In [93]:
```
import pandas as pd

results_data = {
    'Model': [
        'Random Forest (IQR Capping)',
        'XGBoost (IQR Capping)',
        'Random Forest (99 Percentile Capping)',
        'XGBoost (99 Percentile Capping)',
        'Random Forest (Dropping Outliers - 99th/1st Percentile)',
        'XGBoost (Dropping Outliers - 99th/1st Percentile)',
        'XGBoost (LOF Outlier Removal)'
    ],
    'Test RMSE': [
        2.9472879185684406,
        2.568160602928535,
        2.8581816004847944,
        2.460683980359667,
        2.776055728831601,
        2.3131270561945074,
        3.0191399719353984
    ],
    'Test MAE': [
        2.156438624474648,
        1.903486565852385,
        2.0923811646276524,
        1.844360317165554,
        2.066352046614049,
        1.7576655625180457,
        2.296897917321679
    ],
    'Test R^2': [
        0.9007565907410954,
        0.9246469377861838,
        0.906666793702913,
        0.930821963516015,
        0.9029134390147133,
        0.932593506724027,
        0.8955527175585825
    ],
    'Test MAPE': [
        0.04656694190843427,
        0.04103379376239669,
        0.04530804403831575,
        0.03988382439175553,
        0.04553571950084117,
        0.038694048548649135,
        0.0495936054684668
    ],
    'Test Accuracy': [
        0.9534330580915658,
        0.9589662062376033,
        0.9546919559616842,
        0.9601161756082445,
```

```
        0.9544642804991589,
        0.9613059514513509,
        0.9504063945315332
    ]
}

results_df = pd.DataFrame(results_data)

# Format numerical columns to 2 decimal places
formatted_results_df = results_df.round(2)

display(formatted_results_df)
```

	Model	Test RMSE	Test MAE	Test R^2	Test MAPE	Test Accuracy
0	Random Forest (IQR Capping)	2.95	2.16	0.90	0.05	0.95
1	XGBoost (IQR Capping)	2.57	1.90	0.92	0.04	0.96
2	Random Forest (99 Percentile Capping)	2.86	2.09	0.91	0.05	0.95
3	XGBoost (99 Percentile Capping)	2.46	1.84	0.93	0.04	0.96
4	Random Forest (Dropping Outliers - 99th/1st Pe...	2.78	2.07	0.90	0.05	0.95
5	XGBoost (Dropping Outliers - 99th/1st Percentile)	2.31	1.76	0.93	0.04	0.96
6	XGBoost (LOF Outlier Removal)	3.02	2.30	0.90	0.05	0.95

In [94]:
```
df_lof
```

Out[94]:

	market_id	store_primary_category	order_protocol	total_items	subtota
0	1.0	4	1.0	4	344
1	2.0	46	2.0	1	190
2	2.0	36	3.0	4	477
3	1.0	38	1.0	1	152
4	1.0	38	1.0	2	362
...	...	...	...	...	.
175772	1.0	28	4.0	3	138
175773	1.0	28	4.0	6	301
175774	1.0	28	4.0	5	183
175775	1.0	58	1.0	1	117
175776	1.0	58	1.0	4	260

175430 rows × 17 columns

In [95]:

```
import pandas as pd

results_data = {
    'Model': [
        'Random Forest (IQR Capping)',
        'XGBoost (IQR Capping)',
        'Random Forest (99 Percentile Capping)',
        'XGBoost (99 Percentile Capping)',
        'Random Forest (Dropping Outliers - 99th/1st Percentile)',
        'XGBoost (Dropping Outliers - 99th/1st Percentile)',
        'XGBoost (LOF Outlier Removal)'
    ],
    'Test RMSE': [
        2.9472879185684406,
        2.568160602928535,
        2.8581816004847944,
        2.460683980359667,
        2.776055728831601,
        2.3131270561945074,
        3.0191399719353984
    ],
    'Test MAE': [
        2.156438624474648,
        1.903486565852385,
        2.0923811646276524,
        1.844360317165554,
        2.066352046614049,
        1.7576655625180457,
        2.296897917321679
```

```
    ],
    'Test R^2': [
        0.9007565907410954,
        0.9246469377861838,
        0.906666793702913,
        0.930821963516015,
        0.9029134390147133,
        0.932593506724027,
        0.8955527175585825
    ],
    'Test MAPE': [
        0.04656694190843427,
        0.04103379376239669,
        0.04530804403831575,
        0.03988382439175553,
        0.04553571950084117,
        0.038694048548649135,
        0.0495936054684668
    ],
    'Test Accuracy': [
        0.9534330580915658,
        0.9589662062376033,
        0.9546919559616842,
        0.9601161756082445,
        0.9544642804991589,
        0.9613059514513509,
        0.9504063945315332
    ]
}

results_df = pd.DataFrame(results_data)

# Format numerical columns to 2 decimal places
formatted_results_df = results_df.round(2)

display(formatted_results_df)
```

	Model	Test RMSE	Test MAE	Test R^2	Test MAPE	Test Accuracy
0	Random Forest (IQR Capping)	2.95	2.16	0.90	0.05	0.95
1	XGBoost (IQR Capping)	2.57	1.90	0.92	0.04	0.96
2	Random Forest (99 Percentile Capping)	2.86	2.09	0.91	0.05	0.95
3	XGBoost (99 Percentile Capping)	2.46	1.84	0.93	0.04	0.96
4	Random Forest (Dropping Outliers - 99th/1st Pe...	2.78	2.07	0.90	0.05	0.95
5	XGBoost (Dropping Outliers - 99th/1st Percentile)	2.31	1.76	0.93	0.04	0.96
6	XGBoost (LOF Outlier Removal)	3.02	2.30	0.90	0.05	0.95

In [95]:

In [95]:

In [95]:

Neural Network

In [ ]:

Neural Network Architecture, Hyperparameters, and Tuning

To model delivery time predictions, a neural network provides a flexible way to learn complex, non-linear relationships between input features and the target variable. The architecture consists of an input layer, multiple hidden layers, and an output layer. Each neuron applies a weighted transformation followed by an activation function, allowing the model to capture subtle interactions in the data. During training, the network updates its weights through backpropagation to minimize prediction error. Over several iterations, the model learns how different operational factors influence delivery time

Neural Network Architecture

A neural network (NN) consists of interconnected layers of nodes (neurons). The basic architecture includes:

- **Input Layer**: Receives the initial data. The number of neurons typically matches the number of features in the dataset.
- **Hidden Layers**: One or more layers between the input and output layers. These layers perform non-linear transformations on the data, extracting increasingly complex features. The number of hidden layers and neurons within each is a key design choice.
- **Output Layer**: Produces the final prediction. The number of neurons and activation function depend on the task (e.g., one neuron with linear activation for regression, multiple neurons with softmax for multi-class classification).

Common Hyperparameters

Hyperparameters are configuration settings external to the model that are optimized during the training process. Key hyperparameters for neural networks include:

- **Number of Hidden Layers**: Determines the depth of the network. Deeper networks can learn more complex patterns but are harder to train.
- **Number of Neurons (Units) per Layer**: Controls the width of each layer. More neurons per layer increase the model's capacity.
- **Activation Functions**: Introduce non-linearity into the network, allowing it to learn complex mappings. Common choices include ReLU (Rectified Linear Unit), Sigmoid, and Tanh.
- **Learning Rate**: Determines the step size at each iteration while moving toward a minimum of the loss function. A smaller learning rate can lead to better convergence but slower training; a larger learning rate might converge faster but risk overshooting the minimum.
- **Batch Size**: The number of samples processed before the model's internal parameters are updated. Smaller batch sizes introduce more noise but can help escape local minima, while larger batch sizes provide a more stable gradient.
- **Epochs**: The number of complete passes through the entire training dataset. Too few epochs can lead to underfitting, while too many can

lead to overfitting.
- **Optimizer**: An algorithm used to update model weights and minimize the loss function (e.g., Adam, SGD, RMSprop).
- **Regularization (L1/L2)**: Techniques to prevent overfitting by adding a penalty to the loss function based on the magnitude of the weights.
- **Dropout**: A regularization technique where a percentage of neurons in a layer are randomly ignored during training, forcing the network to learn more robust features.
- **Batch Normalization**: A technique to normalize the inputs of each layer, stabilizing and accelerating the training process.

Hyperparameter Tuning

Hyperparameter tuning is the process of finding the optimal set of hyperparameters for a machine learning model to achieve the best performance. It's an iterative process because the ideal hyperparameters are often dataset-dependent.

Overfitting and Underfitting

- **Overfitting**: Occurs when a model learns the training data too well, capturing noise and specific patterns that do not generalize to new, unseen data. This results in excellent performance on the training set but poor performance on the validation/test set. Symptoms include low training loss but high validation loss.
- **Underfitting**: Occurs when a model is too simple to capture the underlying patterns in the training data. It performs poorly on both the training and validation/test sets. Symptoms include high training and validation loss.

Effective hyperparameter tuning aims to find a balance that minimizes both underfitting and overfitting.

Types of Hyperparameter Tuning

Several strategies exist for hyperparameter tuning:

1. **Manual Search**: Adjusting hyperparameters based on intuition and prior experience, then evaluating the model's performance. This can be time-consuming and inefficient for complex models.
2. **Grid Search**: Exhaustively searching through a predefined subset of

the hyperparameter space. It tests every possible combination of specified hyperparameter values. While thorough, it becomes computationally expensive as the number of hyperparameters and their possible values increase.

3. **Random Search**: Randomly sampling hyperparameters from a predefined distribution for a fixed number of iterations. This is often more efficient than grid search, especially when only a few hyperparameters significantly impact performance.
4. **Bayesian Optimization**: A more advanced technique that builds a probabilistic model of the objective function (e.g., validation loss vs. hyperparameters) and uses it to select the most promising hyperparameters to evaluate in the real model. It's more efficient than grid or random search for complex spaces.
5. **Gradient-based Optimization**: Directly computing gradients with respect to hyperparameters (when possible) to find optimal values. Less common for traditional neural network hyperparameters.

In [96]:
```
df_2.head(5)
```

Out[96]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_pr
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	

Data Preprocessing for Neural Network

In [97]:
```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np

df_nn = df_2.copy()
```

```
columns_to_drop = ['created_at', 'actual_delivery_time']
df_nn = df_nn.drop(columns=columns_to_drop)

# 3. Separate the df_nn DataFrame into features (x_nn) and the target variable
x_nn = df_nn.drop('delivery_time', axis=1)
y_nn = df_nn['delivery_time']

# 4. Apply the same feature engineering steps to x_nn for time-based features
labels1 = ['Morning', 'Afternoon', 'Evening', 'Night']
bins1 = [0, 6, 12, 18, 24]
x_nn['create_hour'] = pd.cut(x_nn['create_hour'], bins=bins1, labels=labels1,

labels3 = ['0-15', '15-30', '30-45', '45-60']
bins3 = [0, 15, 30, 45, 60]
x_nn['create_min'] = pd.cut(x_nn['create_min'], bins=bins3, labels=labels3, ri

day_map = {
    0: 'Monday',
    1: 'Tuesday',
    2: 'Wednesday',
    3: 'Thursday',
    4: 'Friday',
    5: 'Saturday',
    6: 'Sunday'
}
x_nn['create_dayofweek'] = x_nn['create_dayofweek'].map(day_map)

# 5. Define the lists of column names for preprocessing
categorical_cols_nn = ['market_id', 'store_primary_category', 'order_protocol'
numerical_cols_nn = ['total_items', 'subtotal', 'num_distinct_items', 'min_ite
boolean_cols_nn = ['is_weekend']

# Ensure categorical columns are treated as strings before OneHotEncoding
for col in categorical_cols_nn:
    x_nn[col] = x_nn[col].astype(str)

# 6. Create a ColumnTransformer named preprocessor_nn
preprocessor_nn = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols_nn),
        ('num', StandardScaler(), numerical_cols_nn),
        ('bool', 'passthrough', boolean_cols_nn)
    ])
X_train_nn, X_test_nn, y_train_nn, y_test_nn = train_test_split(x_nn, y_nn, te

pipeline_nn = Pipeline(steps=[('preprocessor', preprocessor_nn)])
X_train_nn = pipeline_nn.fit_transform(X_train_nn)
X_test_nn = pipeline_nn.transform(X_test_nn)

print("Data preprocessing for Neural Network complete. X_train_nn, X_test_nn,
```

```
Data preprocessing for Neural Network complete. X_train_nn, X_test_nn, y_trai
n_nn, y_test_nn are created.
```

Baseline Model

In [98]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

input_shape = X_train_nn.shape[1]
print(f"Input shape for the neural network: {input_shape}")

model = Sequential()
model.add(Dense(128, activation='relu', input_shape=(input_shape,)))
model.add(Dense(64, activation='relu'))
model.add(Dense(1, activation='linear'))
model.compile(optimizer=Adam(), loss='mean_squared_error', metrics=['mean_absc
# loss for regression model would be onlyy MSE i.e when  the model tries to mi
model.summary()
```

```
Input shape for the neural network: 110
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: User
Warning: Do not pass an `input_shape`/`input_dim` argument to a layer. When usi
ng Sequential models, prefer using an `Input(shape)` object as the first layer
in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	14,208
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 1)	65

Total params: 22,529 (88.00 KB)

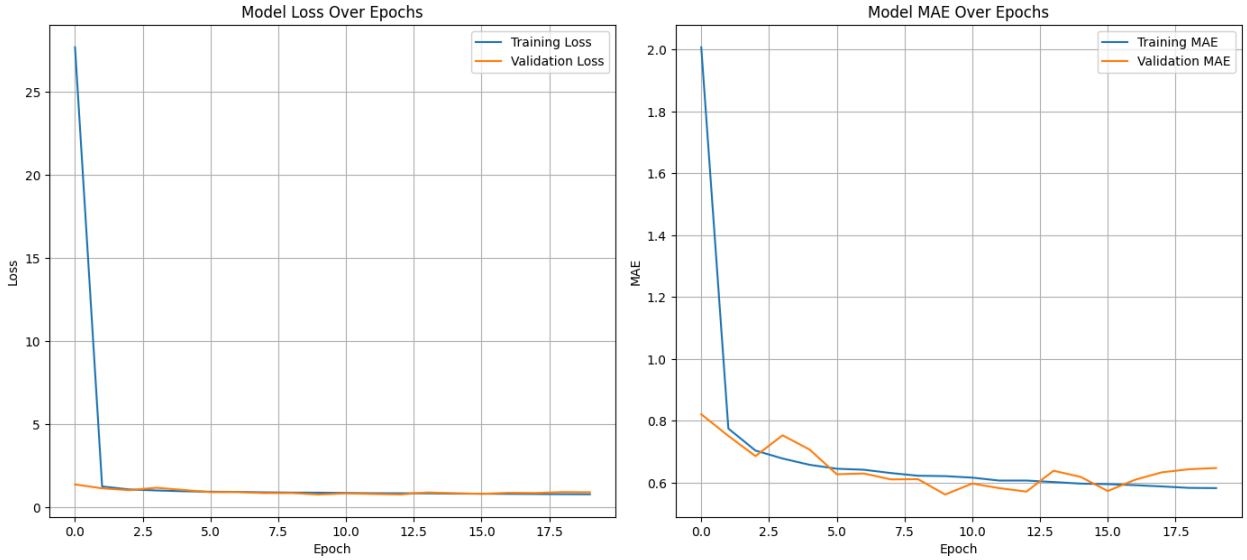
Trainable params: 22,529 (88.00 KB)

Non-trainable params: 0 (0.00 B)

In [99]:
```
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score,

history = model.fit(X_train_nn, y_train_nn,
                    epochs=20,
                    batch_size=32,
                    validation_data=(X_test_nn, y_test_nn),
                    verbose=0)

print("Model training complete.")

loss, mae = model.evaluate(X_test_nn, y_test_nn, verbose=0)
print(f"\nTest Loss (MSE): {loss:.4f}")
print(f"Test MAE: {mae:.4f}")
```

```python
y_pred_nn = model.predict(X_test_nn)

rmse_nn = np.sqrt(mean_squared_error(y_test_nn, y_pred_nn))
mae_nn = mean_absolute_error(y_test_nn, y_pred_nn)
r2_nn = r2_score(y_test_nn, y_pred_nn)
mape_nn = mean_absolute_percentage_error(y_test_nn, y_pred_nn)

print(f"Test RMSE: {rmse_nn:.4f}")
print(f"Test R^2: {r2_nn:.4f}")
print(f"Test MAPE: {mape_nn:.4f}")

plt.figure(figsize=(14, 12))

# Model Loss
plt.subplot(2, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Model MAE
plt.subplot(2, 2, 2)
plt.plot(history.history['mean_absolute_error'], label='Training MAE')
plt.plot(history.history['val_mean_absolute_error'], label='Validation MAE')
plt.title('Model MAE Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('MAE')
plt.legend()
plt.grid(True)



plt.tight_layout()
plt.show()
```

```
Model training complete.

Test Loss (MSE): 0.9220
Test MAE: 0.6476
1099/1099 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step
Test RMSE: 0.9602
Test R^2: 0.9895
Test MAPE: 0.0141
```



In [100…
```
history.history['mean_absolute_error']
```

Out[100…
```
[2.0072829723358154,
 0.7750351428985596,
 0.7038825750350952,
 0.6781455278396606,
 0.657806396484375,
 0.6451509594917297,
 0.6418683528900146,
 0.6311506628990173,
 0.6225172877311707,
 0.6212645769119263,
 0.6163548827171326,
 0.6068828105926514,
 0.6069051623344421,
 0.6021372675895691,
 0.5970501899719238,
 0.5952993631362915,
 0.5920302867889404,
 0.587920069694519,
 0.5832424163818359,
 0.5824934840202332]
```

When to Use Mean Absolute Error (MAE):

- **Interpretability:** We can use MAE when we need a clear, intuitive average error in the original units of our target variable (e.g., "predictions are off by X minutes on average"). It's easy to explain to non-technical audiences.
- **Robustness to Outliers:** Prefer MAE when we don't want large errors (outliers) to disproportionately influence our error metric or inshort when we dont care about much of large error or its not impacting our sales. MAE treats all errors linearly.

When to Use Root Mean Squared Error (RMSE):

- **Penalizing Large Errors:** We can use RMSE when large prediction errors are significantly more and costly than smaller errors. The squaring of errors amplifies the impact of outliers.
- **Detecting Outliers/Systemic Issues:** A much higher RMSE compared to MAE often indicates that our model is making some notably large mistakes.RMSE can be used when we want to emphasize more on large prediction error and large prefiction error affecting the business.This can signal the presence of outliers or specific scenarios where the model performs poorly, which might require further investigation.

In [100…

In [100…

In [100…

Hyperparameter tunning with Keras tuner

In [101…

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l2
# search space or neural network architecture
def build_model(hp):
    model = Sequential()

    for i in range(hp.Int('num_layers', 1, 3)):
        model.add(Dense(
            units=hp.Int(f'units_{i}', min_value=64, max_value=128, step=64),
            activation=hp.Choice('activation', ['relu', 'tanh']),
            kernel_regularizer=l2(hp.Float('l2_reg', 1e-5, 1e-3, sampling='log
        ))

        if hp.Boolean('batch_norm'):
            model.add(BatchNormalization())

        if hp.Boolean('use_dropout'):
            model.add(Dropout(rate=hp.Choice('dropout_rate', [0.2, 0.3])))

    model.add(Dense(1, activation='linear'))

    lr = hp.Float('learning_rate', 1e-4, 1e-2, sampling='log')
    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='mean_squared_error',
        metrics=['mean_absolute_error']
    )
```

```
    return model
```

In [102…
```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	14,208
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 1)	65

Total params: 67,589 (264.02 KB)
Trainable params: 22,529 (88.00 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 45,060 (176.02 KB)

In [103…
```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceL

callback = [
    EarlyStopping(
        monitor='val_loss',
        patience=5,
        restore_best_weights=True
    ),
    ModelCheckpoint(
        filepath='best_model.h5',
        monitor='val_loss',
        save_best_only=True,     # only keep the best model
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,              # reduce LR by half
        patience=3,              # if no improvement for 3 epochs
        min_lr=1e-6
    )
]
```

In [104…
```
!pip install keras-tuner

import keras_tuner as kt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam

# Random search tuner(looking for all the combinations among search space)
tuner = kt.RandomSearch(
    build_model,
```

```
    objective='val_loss',
    max_trials=7,
    executions_per_trial=1,
    directory='tuner_results',
    project_name='delivery_time_nn'
)
# training of the model
tuner.search(X_train_nn, y_train_nn,
             epochs=7,
             batch_size=32,
             validation_data=(X_test_nn, y_test_nn),
             callbacks = callback,
             verbose=1)

best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
print("Best hyperparameters:")
print(f"Layers: {best_hps.get('num_layers')}")
for i in range(best_hps.get('num_layers')):
    print(f"Units in layer {i}: {best_hps.get(f'units_{i}')}")
print(f"Activation: {best_hps.get('activation')}")
print(f"Learning rate: {best_hps.get('learning_rate')}")
print(f"Dropout: {best_hps.get('use_dropout')}, rate: {best_hps.get('dropout_r
print(f"BatchNorm: {best_hps.get('batch_norm')}")

# best hyperparmeter is found out above by tuner using val loss
# and those hyperparmeter (best) is again train on full dataset
# because KerasTuner doesn't usually save the weights of all these trial(diffe
best_model = tuner.hypermodel.build(best_hps)
history = best_model.fit(X_train_nn, y_train_nn,
                         epochs=7,
                         batch_size=32,
                         validation_data=(X_test_nn, y_test_nn),
                         callbacks= callback,
                         verbose=1)

# Evaluate
loss, mae = best_model.evaluate(X_test_nn, y_test_nn, verbose=0)
print(f"\nBest Model Test Loss (MSE): {loss:.4f}")
print(f"Best Model Test MAE: {mae:.4f}")
```

```
Trial 7 Complete [00h 01m 35s]
val_loss: 0.9009143114089966

Best val_loss So Far: 0.9009143114089966
Total elapsed time: 00h 12m 12s
Best hyperparameters:
Layers: 3
Units in layer 0: 64
Units in layer 1: 128
Units in layer 2: 128
Activation: relu
Learning rate: 0.00042727388425869777
Dropout: False, rate: N/A
BatchNorm: False
Epoch 1/7
4385/4395 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 175.7615 - mean_absolute_err
or: 6.1682
Epoch 1: val_loss improved from inf to 1.82116, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
eras.saving.save_model(model)`. This file format is considered legacy. We recom
mend using instead the native Keras format, e.g. `model.save('my_model.keras')`
or `keras.saving.save_model(model, 'my_model.keras')`.
[illegible]
4395/4395 ━━━━━━━━━━━━━━━━━━━━ 13s 3ms/step - loss: 175.4176 - mean_absolute_er
ror: 6.1589 - val_loss: 1.8212 - val_mean_absolute_error: 0.9570 - learning_rat
e: 4.2727e-04
Epoch 2/7
4380/4395 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.7148 - mean_absolute_erro
r: 0.9171
Epoch 2: val_loss improved from 1.82116 to 1.17050, saving model to best_mode
l.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
eras.saving.save_model(model)`. This file format is considered legacy. We recom
mend using instead the native Keras format, e.g. `model.save('my_model.keras')`
or `keras.saving.save_model(model, 'my_model.keras')`.
[illegible]
4395/4395 ━━━━━━━━━━━━━━━━━━━━ 11s 3ms/step - loss: 1.7142 - mean_absolute_erro
r: 0.9169 - val_loss: 1.1705 - val_mean_absolute_error: 0.7268 - learning_rate:
4.2727e-04
Epoch 3/7
4369/4395 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.2310 - mean_absolute_erro
r: 0.7427
Epoch 3: val_loss improved from 1.17050 to 1.00541, saving model to best_mode
l.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
eras.saving.save_model(model)`. This file format is considered legacy. We recom
mend using instead the native Keras format, e.g. `model.save('my_model.keras')`
or `keras.saving.save_model(model, 'my_model.keras')`.
```

```
[illegible]
4395/4395 ━━━━━━━━━━━━━━━━━━━━ 11s 3ms/step - loss: 1.2307 - mean_absolute_erro
r: 0.7426 - val_loss: 1.0054 - val_mean_absolute_error: 0.6384 - learning_rate:
4.2727e-04
Epoch 4/7
4392/4395 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.0608 - mean_absolute_erro
r: 0.6781
Epoch 4: val_loss did not improve from 1.00541
4395/4395 ━━━━━━━━━━━━━━━━━━━━ 11s 3ms/step - loss: 1.0608 - mean_absolute_erro
r: 0.6781 - val_loss: 1.1161 - val_mean_absolute_error: 0.7193 - learning_rate:
4.2727e-04
Epoch 5/7
4378/4395 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.0317 - mean_absolute_erro
r: 0.6590
Epoch 5: val_loss improved from 1.00541 to 0.91624, saving model to best_mode
l.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
eras.saving.save_model(model)`. This file format is considered legacy. We recom
mend using instead the native Keras format, e.g. `model.save('my_model.keras')`
or `keras.saving.save_model(model, 'my_model.keras')`.
[illegible]
4395/4395 ━━━━━━━━━━━━━━━━━━━━ 11s 2ms/step - loss: 1.0317 - mean_absolute_erro
r: 0.6590 - val_loss: 0.9162 - val_mean_absolute_error: 0.6074 - learning_rate:
4.2727e-04
Epoch 6/7
4389/4395 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.9790 - mean_absolute_erro
r: 0.6416
Epoch 6: val_loss did not improve from 0.91624
4395/4395 ━━━━━━━━━━━━━━━━━━━━ 21s 3ms/step - loss: 0.9790 - mean_absolute_erro
r: 0.6416 - val_loss: 1.1458 - val_mean_absolute_error: 0.7326 - learning_rate:
4.2727e-04
Epoch 7/7
4388/4395 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.9567 - mean_absolute_erro
r: 0.6370
Epoch 7: val_loss did not improve from 0.91624
4395/4395 ━━━━━━━━━━━━━━━━━━━━ 12s 3ms/step - loss: 0.9567 - mean_absolute_erro
r: 0.6370 - val_loss: 0.9543 - val_mean_absolute_error: 0.6337 - learning_rate:
4.2727e-04

Best Model Test Loss (MSE): 0.9162
Best Model Test MAE: 0.6074
```

In [110…

```
#predict
y_pred = best_model.predict(X_test_nn)
y_pred
```

```
1099/1099 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step
```

Out[110…
```
array([[50.37523 ],
       [69.45388 ],
       [42.05224 ],
       ...,
       [31.367273],
       [35.66313 ],
       [42.48684 ]], dtype=float32)
```

In [111…
```
y_test_nn.shape,y_pred.shape
```

Out[111…
```
((35156,), (35156, 1))
```

In [112…
```
from sklearn.metrics import mean_absolute_percentage_error
mean_absolute_percentage_error(y_test_nn, y_pred)
```

Out[112…
```
0.012935074576720315
```

In [113…
```
#  summary of the top trials
tuner.results_summary(num_trials=12)
```

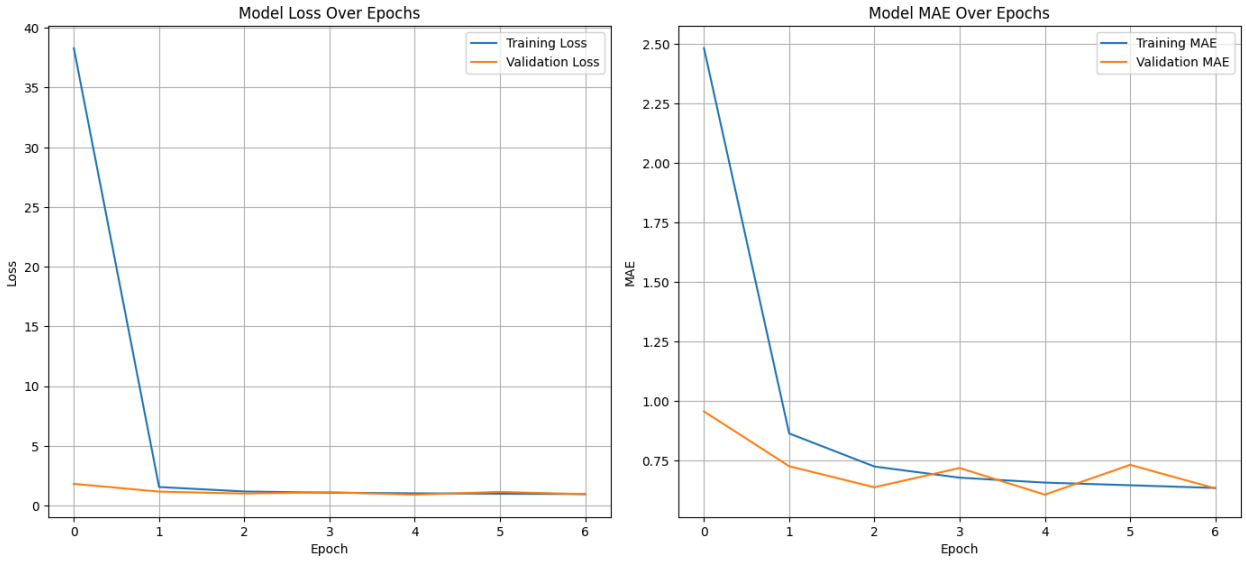
```
Results summary
Results in tuner_results/delivery_time_nn
Showing 12 best trials
Objective(name="val_loss", direction="min")

Trial 6 summary
Hyperparameters:
num_layers: 3
units_0: 64
activation: relu
l2_reg: 0.00010882806869262302
batch_norm: False
use_dropout: False
learning_rate: 0.00042727388425869777
units_1: 128
units_2: 128
dropout_rate: 0.2
Score: 0.9009143114089966

Trial 5 summary
Hyperparameters:
num_layers: 2
units_0: 64
activation: relu
l2_reg: 0.00014888931383669375
batch_norm: True
use_dropout: False
learning_rate: 0.0011196725758620485
units_1: 64
units_2: 128
dropout_rate: 0.2
Score: 1.0662407875061035

Trial 4 summary
Hyperparameters:
num_layers: 3
units_0: 128
activation: relu
l2_reg: 0.00013826038243383546
batch_norm: True
use_dropout: False
learning_rate: 0.0009142311834577992
units_1: 128
units_2: 128
dropout_rate: 0.3
Score: 1.0675040483474731

Trial 2 summary
Hyperparameters:
num_layers: 2
units_0: 64
activation: relu
l2_reg: 2.4908165299495482e-05
batch_norm: True
```

```
use_dropout: False
learning_rate: 0.007764669056757976
units_1: 128
units_2: 128
dropout_rate: 0.2
Score: 1.2281413078308105

Trial 0 summary
Hyperparameters:
num_layers: 3
units_0: 128
activation: tanh
l2_reg: 0.00029580628296941615
batch_norm: True
use_dropout: False
learning_rate: 0.002062409371960664
units_1: 64
units_2: 64
Score: 1.2880041599273682

Trial 3 summary
Hyperparameters:
num_layers: 1
units_0: 64
activation: relu
l2_reg: 2.2482374686240204e-05
batch_norm: True
use_dropout: True
learning_rate: 0.0005334045447934204
units_1: 64
units_2: 128
dropout_rate: 0.3
Score: 1.8542089462280273

Trial 1 summary
Hyperparameters:
num_layers: 2
units_0: 64
activation: tanh
l2_reg: 0.0003218438555994657
batch_norm: True
use_dropout: True
learning_rate: 0.0018754503534125356
units_1: 128
units_2: 128
dropout_rate: 0.2
Score: 2.122019052505493
```

In [107…

```python
plt.figure(figsize=(14, 12))

# Model Loss
plt.subplot(2, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
```

```
plt.title('Model Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Model MAE
plt.subplot(2, 2, 2)
plt.plot(history.history['mean_absolute_error'], label='Training MAE')
plt.plot(history.history['val_mean_absolute_error'], label='Validation MAE')
plt.title('Model MAE Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('MAE')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Model Training Insights

- Rapid initial learning: Both training loss and MAE drop sharply after the first epoch, showing that the neural network quickly captured the underlying patterns in the data.
- Stable validation performance: Validation loss and MAE remain consistently low and flat across epochs, indicating strong generalization and no signs of overfitting within the training window.

In [ ]:

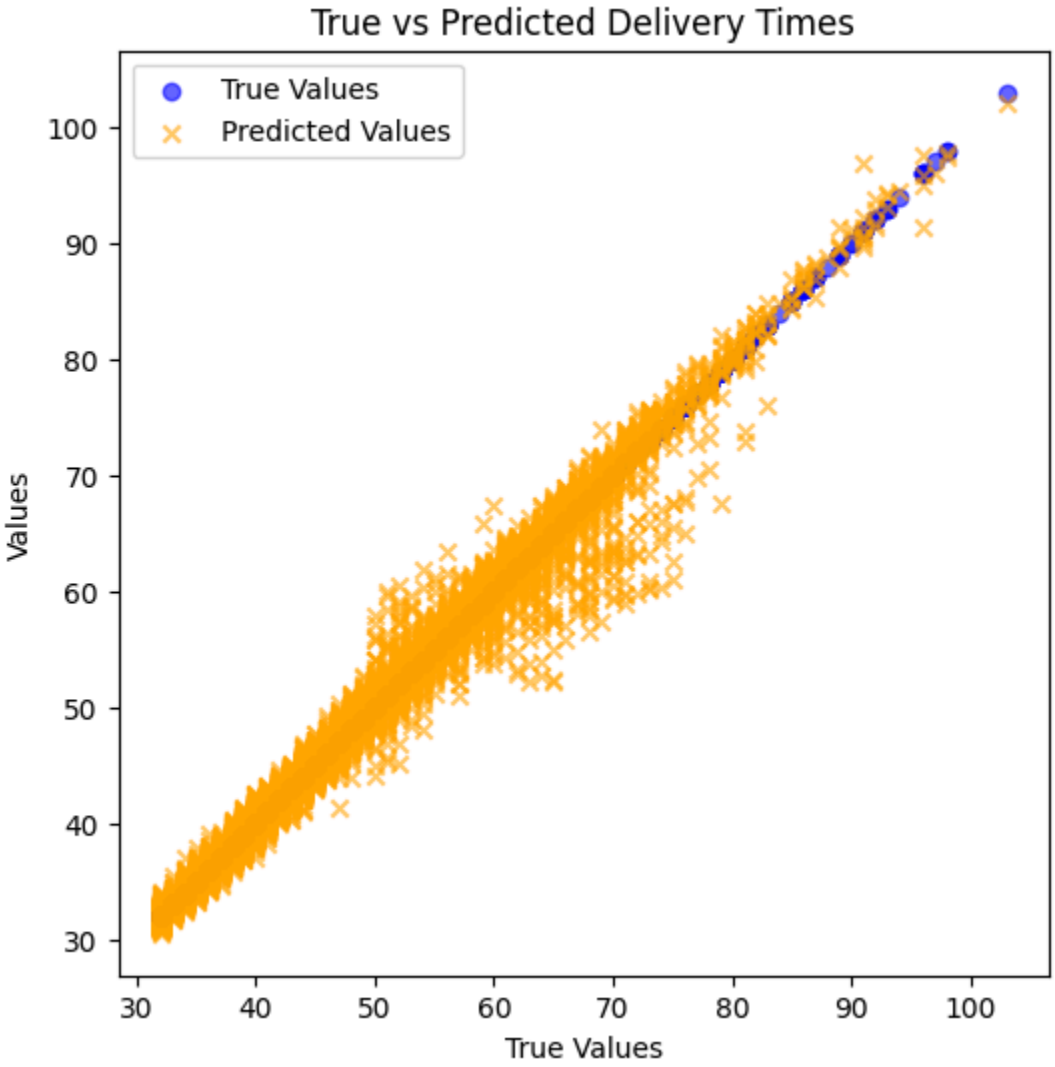
In [117…

```
#plot test mae by epoch (from x_test )
```

Out[117… 0.6074131727218628

Error Analysis

In [114…
```
import matplotlib.pyplot as plt

plt.figure(figsize=(6,6))

plt.scatter(y_test_nn, y_test_nn, color="blue", alpha=0.6, marker="o", label="
plt.scatter(y_test_nn, y_pred, color="orange", alpha=0.6, marker="x", label="P

plt.xlabel("True Values")
plt.ylabel("Values")
plt.title("True vs Predicted Delivery Times")
plt.legend()
plt.show()
```

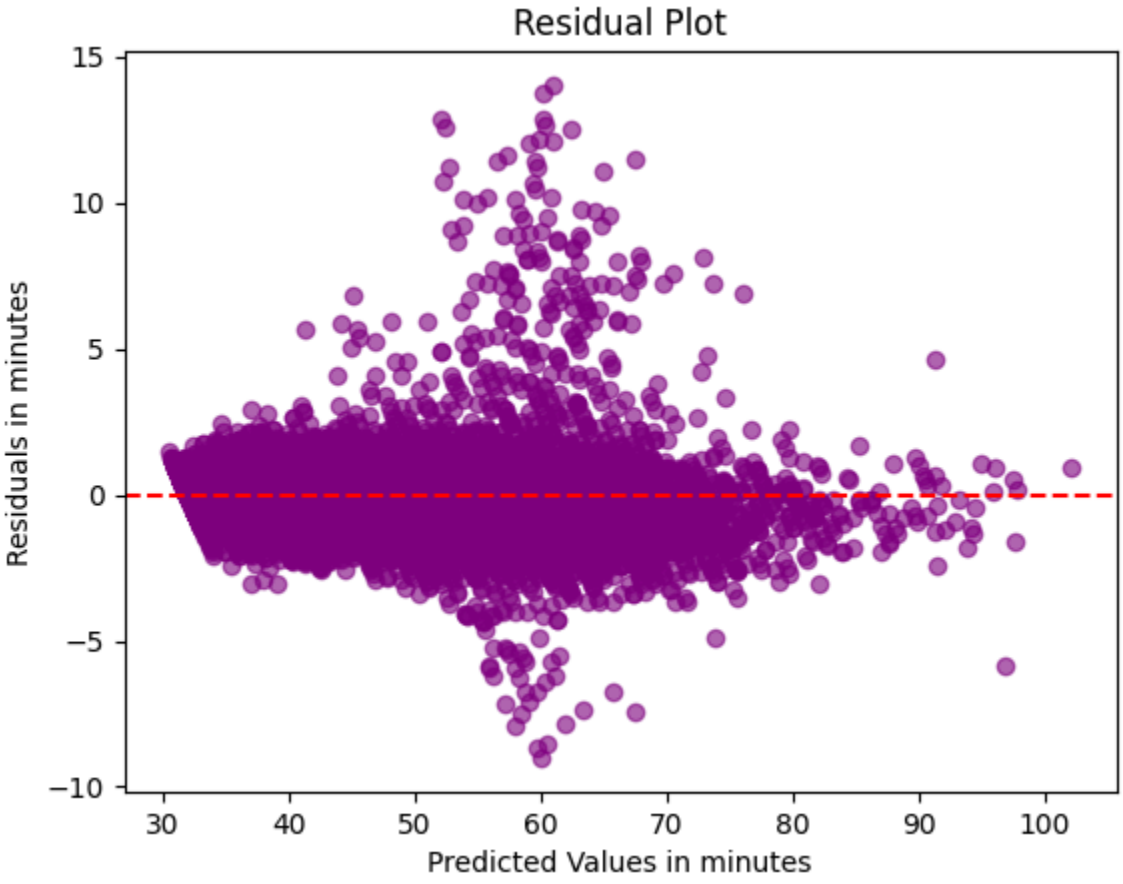


Observations:

- The predicted values (orange crosses) closely follow the true values (blue dots) along the diagonal, indicating a strong correlation and reliable model performance.
- The overlap between true and predicted points suggests that the neural network generalizes well, with minimal systematic bias in delivery time estimation.

In [115…

```
residuals = y_test_nn - y_pred.flatten()
plt.scatter(y_pred, residuals, alpha=0.6, color="purple")
plt.axhline(0, color="red", linestyle="--")
plt.xlabel("Predicted Values in minutes")
plt.ylabel("Residuals in minutes")
plt.title("Residual Plot")
plt.show()
```



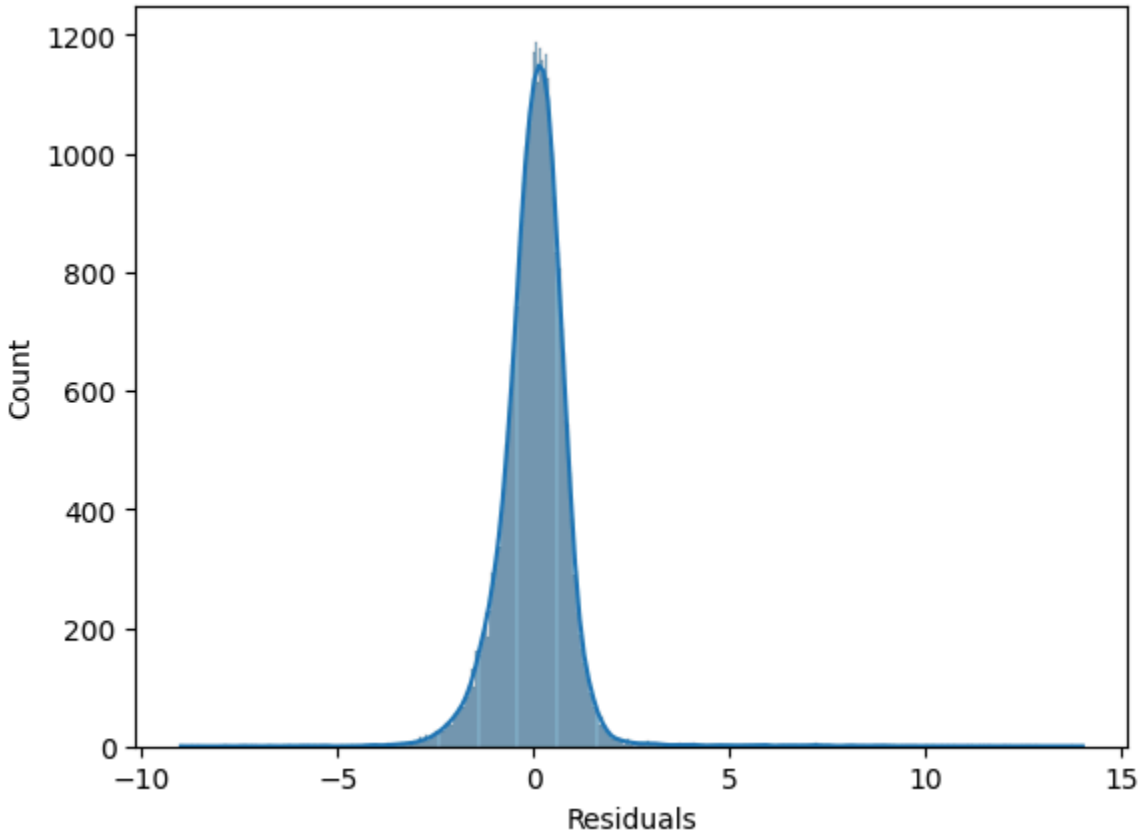
Observations:

- Residuals are concentrated around zero, showing that the model's predictions align closely with actual delivery times.
- Errors appear randomly distributed across the prediction range,

suggesting no systematic bias in over- or underestimation.

In [116…
```
sns.histplot(residuals, kde=True)
plt.xlabel("Residuals")
```

Out[116… `Text(0.5, 0, 'Residuals')`



Business Implication:
Accurate prediction of delivery times enables better resource planning and customer communication, reducing uncertainty and improving operational efficiency.

In [ ]:

Conclusion
The neural network showed clear evidence of effective learning. Training loss and MAE dropped sharply after the first epoch, reflecting rapid adaptation to the data. Validation curves remained stable and low, which confirms that the model generalized well without overfitting. The scatter plot of true vs predicted delivery times further reinforced this, with predictions closely tracking actual values along the diagonal. Residual analysis revealed errors clustered around zero, with only a handful of larger deviations, pointing to isolated outliers rather than systemic bias.

Taken together, these results demonstrate a model that is both accurate and reliable, capable of producing delivery time estimates that can meaningfully support operational planning and customer communication.

In [ ]:

Model Interpretability

In [ ]:

In [119…]:
```
!pip install lime
```

```
Collecting lime
  Downloading lime-0.2.0.1.tar.gz (275 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 275.7/275.7 kB 5.2 MB/s eta 0:00:0
0
  Preparing metadata (setup.py) ... done
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-pac
kages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages
(from lime) (1.16.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(from lime) (4.67.3)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/
dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/
dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-image>=0.12->lime) (3.6.1)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-p
ackages (from scikit-image>=0.12->lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python
3.12/dist-packages (from scikit-image>=0.12->lime) (2.37.2)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.1
2/dist-packages (from scikit-image>=0.12->lime) (2026.3.3)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-
packages (from scikit-image>=0.12->lime) (26.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/di
st-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn>=0.18->lime) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.1
2/dist-packages (from scikit-learn>=0.18->lime) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/di
st-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-p
ackages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/d
ist-packages (from matplotlib->lime) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/d
ist-packages (from matplotlib->lime) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/di
st-packages (from matplotlib->lime) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.1
2/dist-packages (from matplotlib->lime) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packa
ges (from python-dateutil>=2.7->matplotlib->lime) (1.17.0)
Building wheels for collected packages: lime
  Building wheel for lime (setup.py) ... done
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834 sh
a256=8d38a3c85d2ba9e0bd05c88eebb99e445a31e4c1b040c609f03e0da7cb7bdb85
  Stored in directory: /root/.cache/pip/wheels/e7/5d/0e/4b4fff9a47468fed5633211
fb3b76d1db43fe806a17fb7486a
Successfully built lime
```

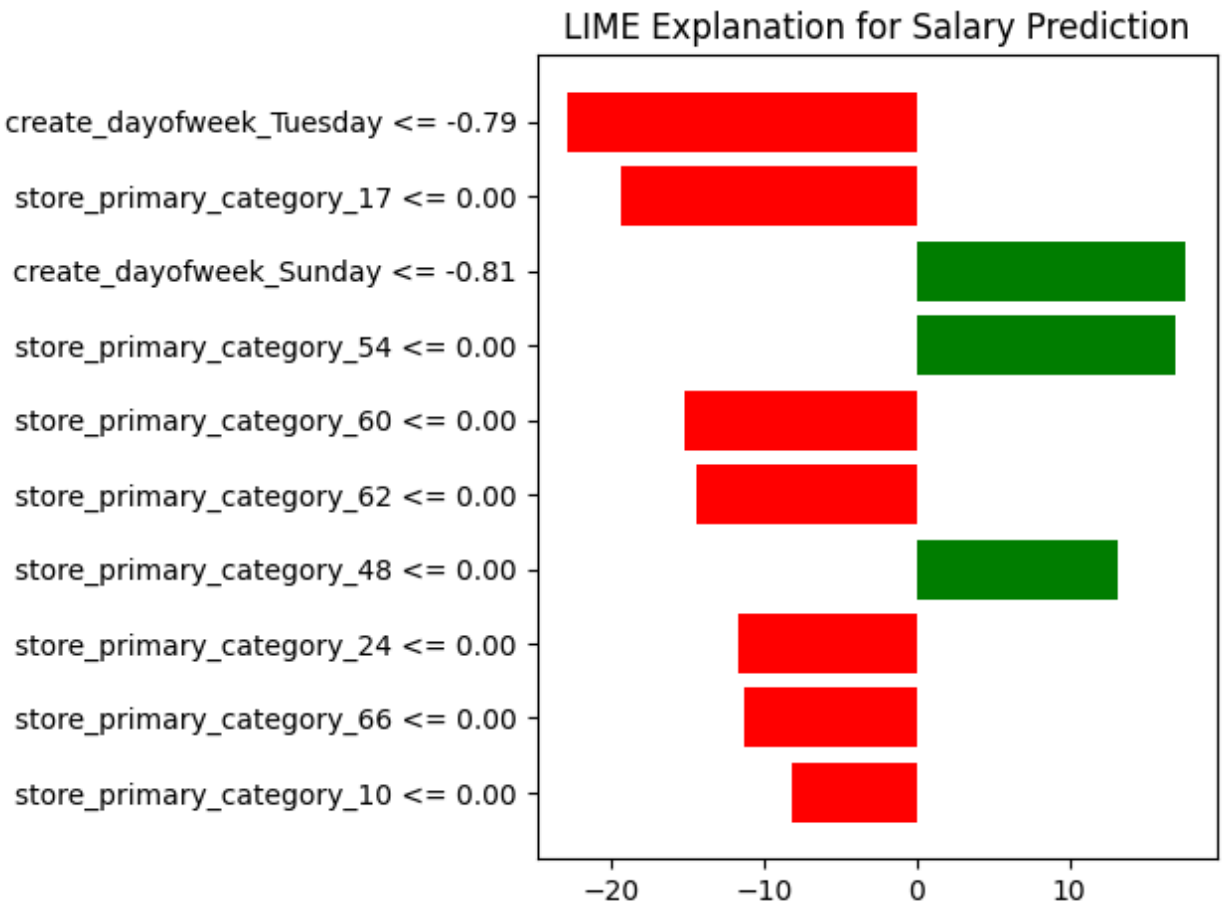
```
Installing collected packages: lime
Successfully installed lime-0.2.0.1
```

In [124…
```python
import lime
import lime.lime_tabular

# Use preprocessed training data (numeric only)
explainer = lime.lime_tabular.LimeTabularExplainer(
    training_data=X_train_nn.toarray(),       # numeric after preprocessing
    feature_names=feature_names,              # reconstructed names (numerical
    mode='regression'
)


def predict_fn(data):
    return best_model.predict(data).ravel()
```

In [125…
```python
# Pick a random test instance (already preprocessed)
instance_idx = np.random.randint(0, X_test_nn.shape[0])
instance_to_explain = X_test_nn[instance_idx].toarray().ravel()

explanation = explainer.explain_instance(
    data_row=instance_to_explain,
    predict_fn=predict_fn,
    num_features=10
)

print(f"True delivery time: {y_test_nn.iloc[instance_idx]:.2f} minutes")
explanation.show_in_notebook(show_all=False)
```

```
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
True delivery time: 48.00 minutes
```

In [129…
```python
fig = explanation.as_pyplot_figure()

plt.title("LIME Explanation for Salary Prediction")
plt.tight_layout()
plt.show()
```



In [128…

```
# all features contribution towards prediction value
explanation.show_in_notebook(show_all=True)
```

Observation on Lime:- LIME breaks down which features pushed the prediction upward (positive, orange) or downward (negative, blue) relative to the baseline.

Negative contributions (lowering predicted time):

create_dayofweek_Tuesday (−22.81 minutes)

store_primary_category_17 (−19.30 minutes)

store_primary_category_60 (−15.18 minutes)

store_primary_category_62 (−14.32 minutes)

Other store categories also contributed negatively.

👉 Interpretation: Orders placed on Tuesday and certain store categories tend to reduce expected delivery times in this model’s logic.

Positive contributions (raising predicted time):

create_dayofweek_Sunday (+17.66 minutes)

store_primary_category_54 (+17.01 minutes)

store_primary_category_48 (+13.20 minutes)

👉 Interpretation: Deliveries on Sunday and orders from specific store categories push the predicted delivery time upward.

Business Intrepretability -

- The model is capturing day-of-week effects: Sundays are associated with longer delivery times (likely due to higher demand or fewer dashers), while Tuesdays are faster.
- Store category also plays a strong role: some categories consistently shorten delivery times, while others lengthen them. This could reflect operational differences (e.g., food prep speed, packaging complexity, or store location density).